

Resistive Crossbars as Approximate Hardware Building Blocks for Machine Learning: Opportunities and Challenges

This article presents a comprehensive overview of the emerging paradigm of approximate computing using NVM crossbars for accelerating machine learning workloads.

BY INDRANIL CHAKRABORTY^{1b}, Graduate Student Member IEEE, MUSTAFA ALI^{1b}, Graduate Student Member IEEE, AAYUSH ANKIT^{1b}, Member IEEE, SHUBHAM JAIN^{1b}, SOURJYA ROY, Graduate Student Member IEEE, SHRIHARI SRIDHARAN, Graduate Student Member IEEE, AMOGH AGRAWAL^{1b}, ANAND RAGHUNATHAN, Fellow IEEE, AND KAUSHIK ROY^{1b}, Fellow IEEE

ABSTRACT | Traditional computing systems based on the von Neumann architecture are fundamentally bottlenecked by data transfers between processors and memory. The emergence of data-intensive workloads, such as machine learning (ML), creates an urgent need to address this bottleneck by designing computing platforms that utilize the principle of colocated memory and processing units. Such an approach, known as “in-memory computing,” can potentially eliminate data movement costs by computing inside the memory array itself. Crossbars

based on resistive nonvolatile memory (NVM) devices have shown immense promise in serving as the building blocks of in-memory computing systems for ML workloads. This is because their high density can lead to higher on-chip storage capacity, while they can also perform massively parallel, *in situ* matrix–vector multiplication (MVM) operations, thereby accelerating the main computational kernel of ML workloads. However, resistive crossbar-based analog computing is inherently approximate due to the device- and circuit-level nonidealities. Furthermore, the area and energy costs of peripheral circuits for conversions between the analog and digital domains can greatly diminish the intrinsic efficiency of crossbar-based MVM computation. We present a comprehensive overview of the emerging paradigm of computing using NVM crossbars for accelerating ML workloads. We describe the design principles of resistive crossbars, including the devices and associated circuits that constitute them. We discuss intrinsic approximations arising from the device and circuit characteristics and study their functional impact on the MVM operation. Next, we present an overview of spatial architectures that exploit the high storage density of NVM crossbars. Furthermore, we elaborate on software frameworks that effectively capture device–circuit–architecture characteristics to evaluate the performance of large-scale deep neural networks (DNNs)

Manuscript received January 26, 2020; revised April 28, 2020; accepted June 13, 2020. Date of publication July 15, 2020; date of current version November 20, 2020. This work was supported in part by the Center for Brain-Inspired Computing Enabling Autonomous Intelligence (C-BRIC), one of six centers in the Joint University Microelectronics Program (JUMP), a Semiconductor Research Corporation (SRC) program sponsored by the Defense Advanced Research Projects Agency (DARPA); in part by the National Science Foundation; in part by Intel; in part by the Office of Naval Research—Multidisciplinary University Research Initiatives (ONR-MURI) Program; and in part by the Vannevar Bush Faculty Fellowship. (Corresponding author: Indranil Chakraborty.)

Indranil Chakraborty, Mustafa Ali, Aayush Ankit, Sourjya Roy, Shrihari Sridharan, Amogh Agrawal, Anand Raghunathan, and Kaushik Roy are with the School of Electrical and Computer Engineering, Purdue University, West Lafayette, IN 47906 USA (e-mail: ichakra@purdue.edu; ali102@purdue.edu; aankit@purdue.edu; roy48@purdue.edu; sridhar4@purdue.edu; agrawa64@purdue.edu; raghunathan@purdue.edu; kaushik@purdue.edu). Shubham Jain is with the IBM Thomas J. Watson Research, Yorktown Heights, NY 10598 USA (e-mail: shubham.jain35@ibm.com).

Digital Object Identifier 10.1109/JPROC.2020.3003007

using resistive crossbar-based hardware. Finally, we discuss open challenges and future research directions that need to be explored in order to realize the vision of resistive crossbars as the building blocks of future computing platforms.

KEYWORDS | Crossbars; deep neural networks (DNNs); machine learning (ML); nonidealities; nonvolatile memories.

I. INTRODUCTION

Rapid advances in machine learning (ML) and artificial intelligence (AI) have touched human lives in an unprecedented way as the application space has proliferated over the last decade [1]. Deep neural networks (DNNs), in particular, have had enormous success in several ML tasks, such as image classification [2], [3], object detection [4], and natural language processing [5]. However, the capabilities of deep learning algorithms have been accompanied by an increase in computational complexity [6]. Designers have proposed special-purpose accelerators [7]–[11] to improve the time and energy required to execute DNNs. These accelerators are designed to efficiently perform matrix–vector multiplication (MVM) operations and the core computational kernel in DNNs. Despite their extensive use of on-chip scratchpads to exploit data reuse, these accelerators are eventually still limited by the off-chip memory bottleneck [12], motivating closer integration of memory and processing units as the next step in the quest for further efficiency. Taking this concept to its logical end leads to a new computing paradigm, “in-memory” computing, wherein memory arrays that store data are also capable of performing compute operations. Although “in-memory” computing can be explored in complementary metal–oxide–semiconductor (CMOS) technology [13]–[18], fundamental design conflicts in CMOS memories as well as their limited density and slowing scaling trends in CMOS are limiting factors. To that end, nonvolatile memory (NVM) technologies [19], [20], such as phase-change memory (PCM) [21], resistive random access memory (RRAM) [22], [23], and spintronics [24], offer immense promise as an alternative to CMOS due to their high storage density and the ability to perform massively parallel *in situ* MVM operations. This led to a growing interest in exploring NVM technology as the substrate for the next generation of ML hardware [19], [25], [26].

The basic computational fabric of NVM technologies is a 2-D cross-point arrangement of NVM devices, commonly known as a crossbar. Due to the intrinsic physics of these NVM devices, they can store multiple states in a highly dense memory with a bit-cell area of $\sim 4F^2$ [27]. Furthermore, due to the resistive nature of NVM devices, crossbars can be used to perform *in situ* MVM operations by programming a matrix of conductances into the 2-D array of NVM devices that constitute the crossbar, applying a vector of input voltages to the rows (columns) of the crossbar, and reading out the currents from the columns (rows) of the crossbar. This mode of computation uses Ohm’s

law and Kirchoff’s laws of current summation to natively realize massively parallel analog MVM operations inside memory and can achieve significantly higher energy efficiency over digital accelerators by eliminating sequential memory accesses. Furthermore, the high density of resistive crossbars can be leveraged to achieve large on-chip storage, thus avoiding frequent data transfer between the off-chip memory and the processor. This promise has fueled great interest over the past decade in designing systems that can leverage the high computing efficiency as well as the high-density storage of crossbar-based MVM units [28]–[30].

The analog nature of computing in NVM crossbars introduces various layers of approximation, which can lead to unique accuracy-energy tradeoffs [32]. The two primary forms of approximations are: 1) functional inaccuracies due to the intrinsic device and circuit nonidealities and 2) approximations made to peripheral circuitry in order to minimize its overheads. Nonidealities signify effects that cause a deviation from ideal MVM computations in resistive crossbars. Undesirable device characteristics and circuit behavior can manifest themselves in three kinds of nonidealities: 1) linear nonidealities; 2) nonlinear nonidealities; and 3) random device and transistor variations. Linear nonidealities arise from parasitic resistances in metal lines, source resistances in input drivers, and sink resistances in sensing circuits. On the other hand, nonlinear nonidealities result from read and write nonlinearities in the NVM devices and the access device characteristics. Moreover, as the NVM process is in its nascent stage of development, device variations are a major challenge. Such variations can appear in the form of intracrossbar and intercrossbar variations. The cumulative effect of the aforementioned nonidealities can cause functional errors at the MVM unit level. In the context of large-scale systems, these errors can accumulate and degrade the application accuracy significantly [33], [34]. Overall, considering the energy benefits of crossbar-based computing systems over their digital counterparts, the approximations due to nonidealities lead to a broader energy accuracy tradeoff across different computing paradigms.

The outputs produced by analog MVM units need to be communicated to other such units for the evaluation of large-scale neural networks. As analog signals are less tolerant of noise, global communication in any large-scale system needs to be digital. As a result, MVM units need peripheral circuits, such as digital-to-analog converters (DACs) at their inputs and analog-to-digital converters (ADCs) at their outputs. These peripheral circuits within MVM units, particularly the ADCs, have been shown to account for a large fraction (up to $\sim 80\%$) of the total energy consumption of the MVM unit [35]. In addition, ADCs consume close to 70% of the area of the MVM unit. Thus, the power and area of peripheral circuits significantly reduce the benefits of the crossbars in terms of compute efficiency and high density. However, as the power consumption of ADCs is a function of its

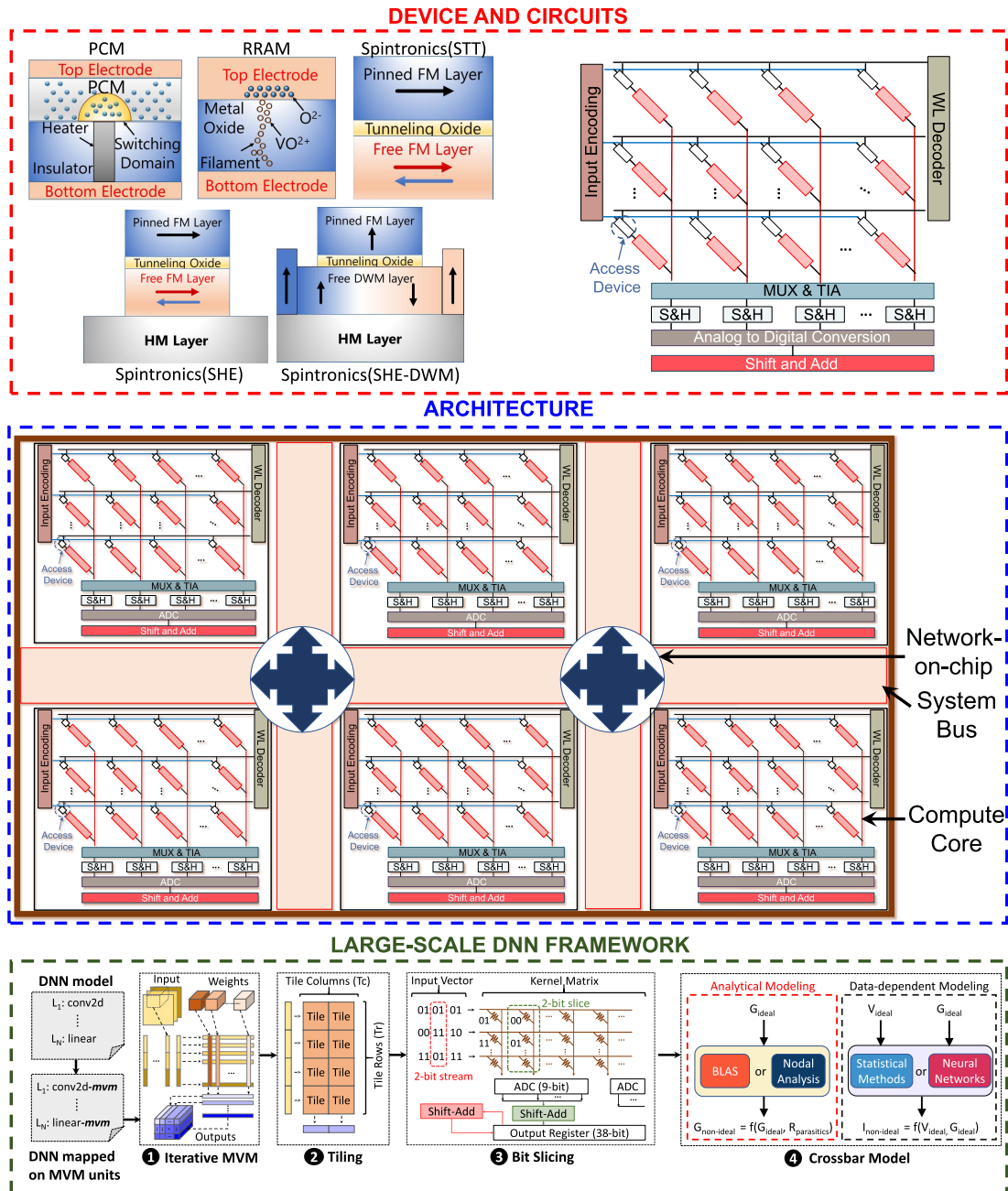


Fig. 1. Different levels of abstractions for resistive crossbar systems for DNN accelerators. First, the basic building blocks of such systems are the NVM devices based on technologies such as PCM, RRAM, and spintronics. These devices can be arranged in a crossbar array, which is augmented with peripheral circuits to enable write operations as well as MVM computation in the analog domain. Such crossbars can be connected to a system bus in a spatial architecture to design large-scale hardware accelerators. Finally, software frameworks allow workloads to be mapped to such hardware fabrics and evaluated considering the impact of nonidealities.

precision [36], we can leverage the error-resiliency of neural networks to strike a balance between the accuracy and energy of NVM crossbar systems by varying the ADC precision. In other words, the ADC can act as an important controlling knob to study energy-accuracy tradeoffs through approximate computing within the NVM crossbar computing paradigm.

In summary, in-memory computing using resistive crossbars presents great promise but also poses several

challenges that need to be analyzed and addressed. In this article, we present a comprehensive overview of resistive crossbars in the context of accelerating ML workloads. We focus on the design of large-scale systems based on resistive crossbars, spanning devices, circuits, and suitable architectures. Fig. 1 describes the different levels of abstractions of NVM crossbar system design from devices to architectures and associated simulation frameworks. In Section II, we provide a detailed overview of the devices

and circuits used to design MVM units based on resistive crossbars. In Section III, we explore several challenges arising from characteristics of the NVM devices as well as circuit-level nonidealities that affect the correct operation of resistive crossbars. Next, in Section IV, we explore spatial architectures that leverage the high density of crossbar-based MVM units to build energy-efficient systems for large-scale DNN inference and training. In Section V, we study the functional aspects of crossbar-based architectures in order to understand the implication of analog computing on large-scale DNN accuracy. Furthermore, we investigate several software- and hardware-based mitigation techniques to recover the degradation in accuracy caused by the nonidealities in crossbar computations. Finally, in Section VI, we conclude this article by providing perspectives on the future of crossbar-based computing systems and the challenges in achieving energy and performance benefits while maintaining application accuracy.

II. ANATOMY OF RESISTIVE CROSSBARS

Let us first introduce the concept of crossbar organization. Fig. 2 shows the conceptual NVM-based crossbar structure, where NVM devices are connected as a matrix similar to a standard memory array. In such resistive crossbars, each memory cell is connected to its corresponding word-line (WL) and bitline (BL). In principle, MVM operations can be efficiently achieved in a resistive crossbar using Ohm's law for multiplication and Kirchhoff's current law (KCL) for summation. The MVM vector operands are applied as analog voltages to WLs of the crossbar. Thus, the resulting current passing through a memory cell is the multiplication of the applied voltage (V) at the WL connected to that cell and the NVM device conductance (G). Furthermore, KCL sums up the currents from all cells connected to the same BL resulting in the multiply-and-accumulate (MAC) operation of V and G . Such inherent MAC computations in crossbars enable parallel MVM operations where the inputs are V and G , while the MAC output is represented as the current passing at BL (I_{BL}). In the following, we discuss, in detail, the basic building blocks of a resistive crossbar, including the NVM cell structure based on various device technologies, selectors, and peripherals required for analog computing.

A. Device Technologies

Device technologies are important to understand the key device parameters that play a key role in large-scale crossbar designs. The scalability of resistive crossbars as an MVM macro is primarily dependent on the following.

- 1) *Resistance ON/OFF ratio*: the ratio between the resistances of the two extreme states in the device.
- 2) *ON-resistance*: the low-resistance state (LRS) of the device.
- 3) *Bit precision*: number of states that can be reliably programmed and read from the device.

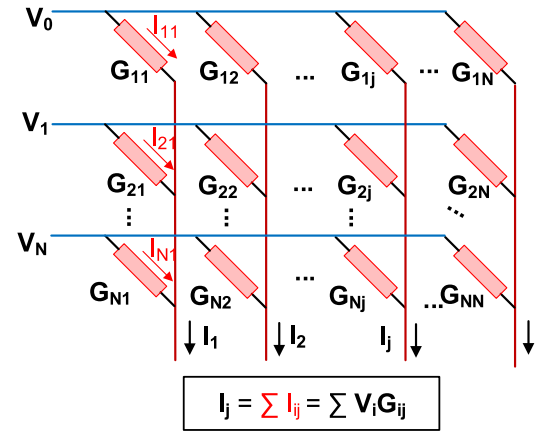


Fig. 2. Crossbar arrangement of NVM devices to enable MVM operations. A voltage vector $V_0, V_1, \dots, V_N$ is applied to each row of the resistive crossbar where the conductance of the devices $G_{11}, \dots, G_{NN}$ form the 2-D matrix. The current output vector $I_1, I_2, \dots, I_N$ in the column represents the MVM result.

These parameters affect the resistive crossbar size, thus playing a role in both the MVM functionality as well as energy and latency. We will study the impact of these parameters on MVM functionality in light of peripheral circuits and device/circuit nonidealities in Section II-D. Energy and latency metrics of crossbars will be discussed in detail in Section IV. Here, we will elucidate different NVM device technologies that have been explored for crossbars.

The basic building block of the MVM compute macro based on crossbars is the NVM devices. However, CMOS memories have also been explored to build this macro, which we will discuss later. NVM devices are mostly two-terminal devices based on phase-change materials (e.g., PCRAM) [37], oxide-based materials (e.g., RRAMs) [38], or magnetic materials [e.g., spin-transfer torque (STT)-MRAM] [39]. On the other hand, some spintronic devices can also be three-terminal devices, such as spin-orbit torque (SOT)-based devices [40], which have decoupled read and write paths. In this section, we will discuss how the inherent physics of these devices enable multibit storage and *in situ* multiplication operations.

1) *Two-Terminal Devices*: Two-terminal devices usually consist of a layer sandwiched between two electrodes. This layer is a phase-change material in the case of a PCM device, an oxide-based material in the case of an RRAM device, and a stack of ferromagnetic (FM) layers in the case of STT-MRAM.

a) *Phase-change materials*: PCMs are chalcogenides that have large contrast in electrical and optical properties resulting in a significantly high ON/OFF-resistance or reflectance ratio. The device structure follows a mushroom cell structure representing the shape of the switching volume above the heater, as shown in Fig. 3. These materials can exist in multiple intermediate states between two extreme resistance states, amorphous (high-resistance),

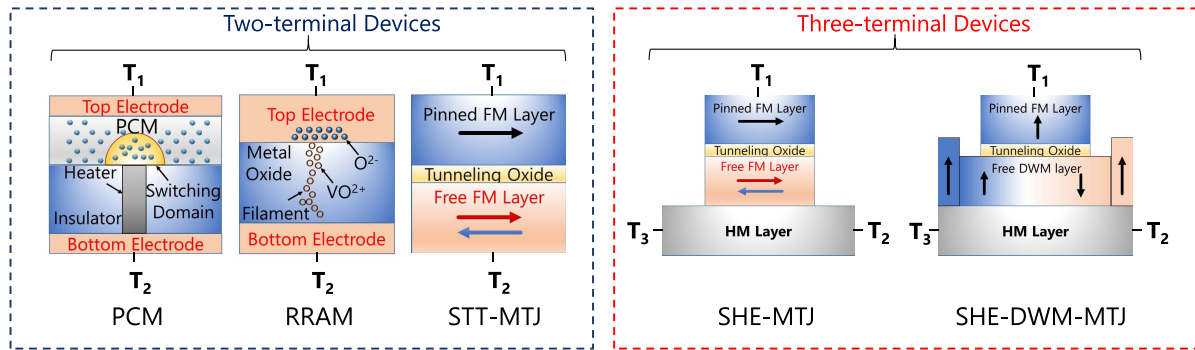


Fig. 3. Different NVM device technologies can be broadly categorized into two types: two-terminal devices and three-terminal devices. Two-terminal devices include PCM, metal-oxide RRAM, and STT-MTJ. Three-terminal devices include SHE-MTJ and SHE-DWM-MTJ.

and crystalline (low-resistance) states. Typically, amorphization involves an abrupt melt-quench process using a high and short voltage pulse and subsequent rapid cooling. Crystallization, on the other hand, is performed by heating the material above its crystallization temperature using a sequence of longer but lower amplitude pulses. Progressive crystallization reduces amorphous thickness gradually and can result in multiple programmable states in PCM devices, which makes it suitable for MVM operations.

There have been several explorations of in-memory computing to perform MVM operations on phase-change memories, both in electronic [41] and photonic domains [42]. In fact, researchers have been successful in experimentally demonstrating large-scale PCM crossbars for ML applications [21], [43]. Table 1 delineates the several advantages and challenges of PCM devices that need to be addressed such as resistance drifting due to structural relaxations after the melt-quench amorphization.

b) Metal-oxide materials and CBRAMs: Perovskite oxides, such as SrTiO₃, and binary metal oxides, such as HfO_x, TiO_x, and TaO_x, exhibit resistive switching properties [48]–[51]. Two-terminal devices based on these materials form the basic building blocks of RRAM-based crossbars. Another class of devices formed using the same structure by simply replacing the oxides by conductive elements constitutes conductive bridge RAMs or CBRAMs [52]. RRAM and CBRAM devices can behave as multilevel memory with two extreme resistance states. The multilevel behavior is due to the soft dielectric breakdown in metal-oxide RRAMs and dissolution of metal ions in CBRAMs. Although switching mechanisms in CBRAMs are primarily filamentary, metal-oxide RRAMs can exhibit both filamentary and nonfilamentary switching. Through filamentary switching, RRAM devices can be programmed to multiple states either by modulating the number of conductive filaments using current compliance or by controlling the degree of oxidation [53]. On the other hand, nonfilamentary switching occurs in oxides of some transition metals, such as PrCa_xPCMO, through modulation of the Schottky barrier by charge trapping or defect migration

at the metal-oxide interface [54]. Switching in CBRAMs is analogous to RRAM filamentary switching except that the filament is a metallic conductive path instead of oxygen vacancies. Typical CBRAMs are based on Ag electrodes where the multilevel behavior is achieved by tuning the volume of Ag-rich and Ag-poor regions [55].

The multilevel memory behavior of RRAMs and CBRAMs is suitable for the aforementioned MVM macro using the crossbar arrangement of these devices. Like PCMs, RRAMs have high density and have been successfully demonstrated to achieve crossbar sizes up to 128 × 128 [56]. RRAM crossbars have also been explored with binary weight cells in a scalable architecture [57]. RRAM technology, however, still needs significant development as it is prone to intercrossbar and intracrossbar variabilities, especially due to the filament formation process [58]. RRAM devices also exhibit nonlinear characteristics [59] that introduce errors in the MVM computations.

c) Spintronic devices: Unlike PCMs and RRAMs, spintronic devices use electron spin as the storage variable. Two-terminal spin devices constitute of a structure known as magnetic tunnel junctions (MTJs), which is basically two FM layers sandwiching a spacer layer (commonly MgO). These ferromagnets encode information in the form of the direction of magnetization and can achieve two stable opposing directions. The direction of the two FMs can either be parallel (P) or antiparallel (AP), which results in two extreme resistance states in these devices. The MTJ can be switched from AP to P or *vice versa* by passing a spin-polarized current, which initiates a phenomenon called STT [60]. Table 1 describes the various advantages and disadvantages of STT MTJs. Although STT-MTJs have been experimentally demonstrated at a large scale for storage class memories [61], their applicability as MVM macros still is at a nascent stage of development. This is primarily due to the low resistance ratio between the P and AP states, commonly quantified as the tunneling magnetoresistance (TMR). Although TMR of 600% has been achieved experimentally for MTJs [46] and theoretical

Table 1 NVM Technologies

Category	NVM Technology	Materials	Switching mechanism	Maximum storage	Advantages	Disadvantages
Two Terminal	PCM	$\text{Ge}_2\text{Sb}_2\text{Te}_5$	Heat driven phase change	3 [44]	- Scalability - High density - High ON/OFF ratio - Multi level storage	- Low endurance - Resistance drifting - High write latency
	RRAM or CBRAM	(HfOx, TiOx, TaOx) or Ag-Si	Filamentary – oxygen migration driven Non-filamentary – Barrier controlled switching	6 [45]	- Scalability - High density - High ON/OFF ratio - Multi level storage	- Variability - Low endurance - High write latency
	Spintronic (STT)	CoFeB–MgO–CoFeB	Spin Transfer Torque	1 [46]	- Low latency - High endurance	- Low density - No multi-level storage - Low ON/OFF ratio
Three Terminal	Spintronic (SHE, DWM)	CoFeB - Mg - CoFeB - Ta	Spin Orbit Torque	> 1 bit per device for DWMs [40]	- Low latency - Low energy - High endurance - Multi level storage	- Low density - Low ON/OFF ratio
Other	FeFET	HfO2	Electrical polarization reversal	3 [47]	- Very high ON/OFF ratio - High endurance - Multi level storage	No scalable demonstration

models predicting that 1000% can be achievable [62], [63], large-scale demonstrations are still lacking.

Two-terminal devices achieve high density and possess a simpler fabrication process. However, due to merged read and write paths, writing into these devices may cause sneak path issues [64]. To address such issues, researchers have proposed using access devices, such as transistors or selectors. An alternative way to circumventing this problem is using three-terminal devices with decoupled read and write paths.

2) Three-Terminal Devices: Typically, three-terminal devices are based on spin devices. By stacking an MTJ on top of the heavy metal (HM) layer, one can write into the FM layer by passing a current through the HM using a phenomenon called the spin Hall effect (SHE) [65]. SHE-MTJs can achieve higher spin injection efficiency than STT-MTJs, thus achieving lower write currents. Moreover, the decoupled read and write paths can address sneak path issues prevalent in crossbars with two-terminal devices. SHE-MTJs have also been explored for achieving multiple states using bigger domain-wall magnets (DWMs) [40]. Here, the position of the domain wall governs the conductance of the SHE-MTJ. Note that the type of domain wall plays a role in the scalability of the device. Devices with perpendicular magnetic anisotropy (PMA) exhibit Neel wall in narrow nanostrips

and Bloch wall in wider nanostrips [66]. While the stability of the Neel wall is aided by the Dzyaloshinskii–Moriya interaction [67], [68], the Bloch walls need external magnetic fields for proper operation [69]. As a result, DWMs with Neel wall can achieve better scalability than those with Bloch walls. Much like STT-MTJs, SHE-MTJs also suffer from significantly low TMR. Moreover, SHE-MTJs based on DWMs have issues with domain-wall stability, which causes variability in the intermediate states, making it unsuitable for large-scale crossbar implementations [70]. Thus, although three-terminal SHE-MTJs offer immense promise in terms of energy efficiency and speed, studies have mostly been limited to simulations [24]. To realize the potential of spin-based crossbars, material exploration to achieve higher TMR is important.

3) Other Emerging NVM Technologies: Besides the popular NVM technologies mentioned earlier, there is another class of materials being explored for NVM devices, namely ferroelectric (FE) field-effect transistors. FE materials exhibit electric-field-driven polarization switching behavior, which is highly nonlinear resulting in hysteresis. Even though FE materials have been explored as a nonvolatile capacitor for a long time [71], recent devices propose integration of FE layer in the gate-stack of a standard transistor [47], [72], [73]. The electrostatic coupling between the FE layer and

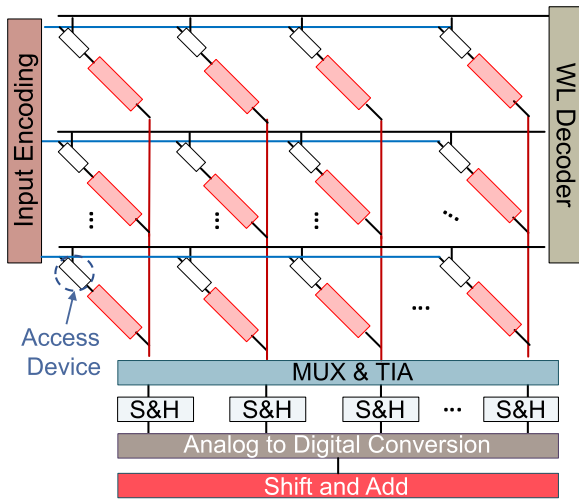


Fig. 4. Resistive crossbar and its peripherals for large-scale integration. Input encoding circuits apply analog voltages to the rows and WL decode activates the access devices. At the output of the crossbar, multiplexing network and the TIA convert analog current to voltage, which is then passed through Sample & Hold (S&H) circuits. The ADC is shared across the columns. The ADC outputs go through the shift and add circuits to account for bit-slicing and bit-streaming.

the underlying transistor, commonly known as the FeFET, allows for the nonvolatile properties of this device. One big advantage of FeFET is its CMOS process compatibility as well as its voltage-based READ/WRITE operation. As a result, FeFETs have shown immense promise as neuromimetic devices [74]–[77]. These properties can be easily ported for ML workloads [78], as discussed in this article.

B. Circuits: Peripherals and Access

The aforementioned MVM unit based on resistive crossbars needs to be supplemented by peripheral circuits and access devices for suitable interfacing between the analog compute unit and digital communication and data flow. Fig. 4 shows the crossbar structure with memory cells and peripheral circuitry, including input encoding and sensing. Let us first consider various device options that provide efficient data access in a memristive cell, including two-terminal selectors and three-terminal selectors (transistors). The access devices are used to avoid sneak paths in resistive crossbars during the write operation. Next, we consider several peripheral circuitry needed for MVM operations based on the type of input encoding. It is worth mentioning that digital inputs can be applied to a memristive crossbar for computing MVM using several encoding schemes that include voltage encoding and time encoding. In voltage encoding, the voltage amplitude is modulated according to the digital input, while the pulsewidth is modulated according to the digital inputs in time encoding. For voltage encoding, the output current has to be converted

to analog voltage using a transimpedance amplifier (TIA). Subsequently, the analog voltage is converted to digital values using ADCs. Shift and add blocks account for weight bit-slicing and input bit-streaming.

1) *Selectors/Access Transistors*: As mentioned earlier, all cells in a crossbar are activated to perform the dot product operation. Therefore, sneak-path currents induced from the partial select operation are avoided during dot-product computing. However, such sneak-path current issue becomes significant during normal read/write operations where only one row is selected and the unselected rows should be simultaneously biased to GND . To that end, selector devices and access transistors have been adopted in practical implementations of memristive crossbars to eliminate the effects of the sneak path problem [79]–[85].

The 2-D selector devices are more dense and suitable for 3-D integration than a transistor; thus, one-selector-one-memristive (1S-1R) device arrays were reported with different types of selector devices, such as in [80]–[82], [84], and [85]. Generally, selector devices can be categorized into two main types: exponential selector (ES) and threshold selector (TS). ES devices maintain an exponential I – V slope to turn on the selector associated with an exponential current increase [81]. ES devices can be implemented by engineering the interface of the oxide/electrode or the bandgap of oxide/oxide with the tunneling current mechanism, such as Ni/TiO₂/Ni. In addition, Cu-containing mixed ionic–electronic conductance (MIEC) offers bidirectional exponential I – V characteristics that can be adopted in 1S-1R crossbars [80]. On the other hand, TS devices rely on the precipitous threshold switching I – V behavior in which the selector conducts current above a threshold voltage and turns off below a hold voltage. Such a threshold switching phenomenon can be implemented in the metal–insulator transition (MIT) materials, such as NbO₂ [82]. Also, the ovonic threshold switch (OTS) based on doped chalcogenide materials has been shown to be a promising TS [85]. Moreover, field-assisted superlinear threshold (FAST) selector has been reported in [84] with steep turn-on slope (<5 mV/decade) and high ON-current J_{ON} (>5 MA/cm²).

Fig. 5 shows I – V characteristics of one-exponential-selector-one-memristive (1ES-1R) device and one-threshold-selector-one-memristive (1TS-1R) device configurations, respectively, where the currents at lower voltages are limited by selectors. At high voltages, the selectors are turned on, and current passes through the memristive device [79], [86].

On the other hand, one-transistor-one-memristive (1T-1R) device crossbars have exhibited MVM computations with fewer errors [88], [89], in comparison to 1S-1R crossbars. Typically, in a 1T-1R crossbar, WL controls the gate of the transistor, while the source-line (SL) is connected to the transistor source. The NVM device top electrode is connected to BL, while its bottom electrode is

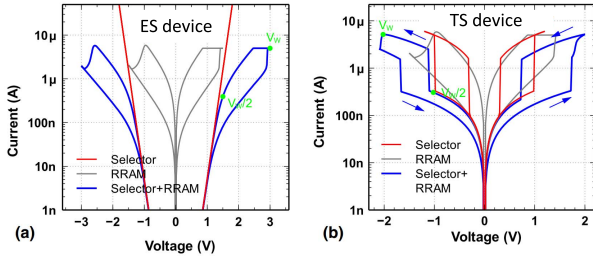


Fig. 5. Representative I - V characteristics for a bipolar RRAM with (a) ES device and (b) TS device. SPICE simulation is performed with the RRAM compact model [86].

connected to the access transistor drain. One disadvantage in the 1T-1R structure is that the cell area is determined by the transistor area, depending on the maximum programming current required to pass across the NVM device. Nevertheless, the 1T-1R structure is standard practice for constructing resistive crossbars as MVM units. Next, let us discuss in detail various peripheral circuit design choices needed for efficient MVM computing in memristive crossbars.

2) *Input Encoding*: In MVM operations of ML workloads, the vector represents the inputs of the neural networks. These input vectors can be encoded in voltage or time in resistive crossbars. For voltage-encoding schemes, WL DACs can be used to convert the digital inputs to analog voltages to be applied at every row. Needless to say that DAC accuracy significantly affects MVM operation accuracy. Moreover, DAC linearity and signal-to-noise and distortion ratio (SNDR) depend heavily on the accuracy of the reference multiplication and division deployed to generate the analog output. Different variants of DAC utilize charge, current, and voltage multiplication or division using switched-capacitor circuits, current steering circuits, and resistor ladders, respectively, to generate the analog output [87], [90]. Fig. 6 shows the schematic of the aforementioned DAC structures.

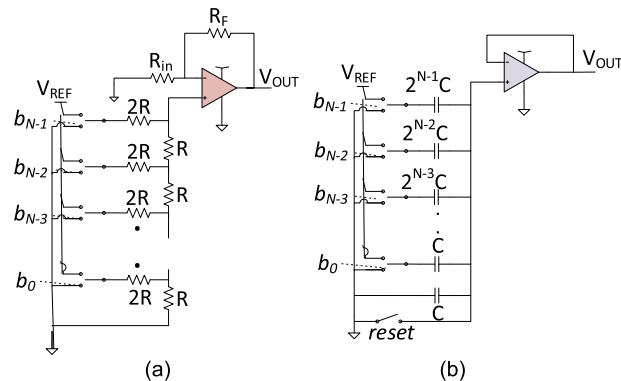


Fig. 6. (a) R-2R ladder-based DAC. (b) Capacitance-based DAC [87].

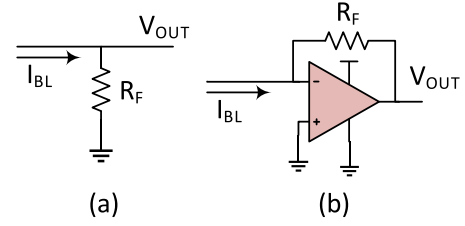


Fig. 7. (a) Resistor-based current-to-voltage converter. (b) Op-amp-based TIA [92].

Regarding time-encoding schemes, WL drivers embed the digital inputs in the pulsewidth of the applied pulses into the crossbar rows using digital-to-time converters [91]. Time-encoding schemes suffer from significant speed reduction compared with commonly adopted voltage encoding since the input information is encoded in time pulses. As such, we consider voltage-encoding scheme throughout the rest of this article. Once the digital inputs are converted to analog voltages applied to each row, analog BL currents carrying the MVM output information are ready to be read out.

3) *Output Sensing*: The MVM output current (I_{BL}) at each crossbar column needs to be converted to the digital domain for transferring data to the next compute unit in the system. To that end, I_{BL} at each column is converted to an analog voltage and then to digital through current-to-voltage converter and ADC, respectively.

a) *Current-to-voltage conversion*: After performing the MVM operations in memristive crossbars, the resulting output current at BLs (I_{BL}) is converted to analog voltage through a current-to-voltage converter. A common circuit technique to convert analog current to voltage is the TIA [92]. In its simplest form, a TIA can be just a single grounded resistance, as shown in Fig. 7(a). Furthermore, a more sophisticated TIA can be an amplifier having an open-loop gain of $-A_0$ with a negative feedback resistance R_F placed around the amplifier, as shown in Fig. 7(b). The fundamental difference between single-resistance and amplifier-based TIAs is that I_{BL} sees a high impedance in Fig. 7(a) and a low impedance in Fig. 7(b). The input impedance of the TIA in Fig. 7(b) can be expressed as follows:

$$R_{in} = \frac{R_F}{1 + A_0}. \quad (1)$$

As such, the circuit is suited for current sensing due to the low input impedance, hence acting as a current-to-voltage converter. The transimpedance gain can be defined as

$$G_{TIA} = \frac{V_{out}}{I_{BL}} = \frac{-I_{BL}R_{in}A_0}{I_{BL}} = -\frac{A_0}{1 + A_0}R_F. \quad (2)$$

If $A_0 \gg 1$, this leads to $G_{TIA} \approx R_F$. The virtual ground provided by the TIA improves the transfer characteristics linearity and bandwidth in two ways.

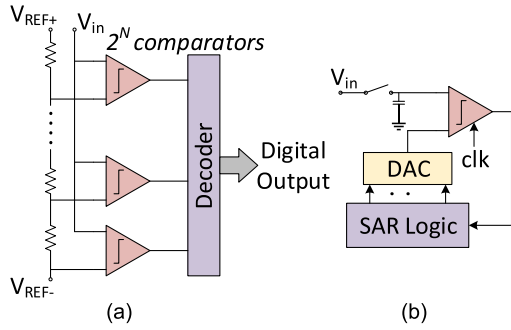


Fig. 8. (a) Flash ADC architecture. (b) SAR ADC architecture [93], [94].

- 1) In the case of the TIA of Fig. 7(b), I_{BL} incurs BL capacitance C_{BL} and TIA input capacitance C_{in} . Such collective capacitance creates a pole at the TIA input equal to $(1 + A_0)/(R_F(C_{BL} + C_{in}))$, while the pole is equal to $1/(R_F C_{BL})$ if using a single resistance as TIA. Therefore, using amplifier-based TIA offers greater bandwidth.
- 2) I_{BL} suffers from nonlinearity for large voltage swings across it, and the amplifier-based TIA reduces these swings by a factor of $1 + A_0$.

b) *Analog-to-digital conversion:* Analog-to-digital conversion is a critical step in the MVM operation using resistive crossbars, as the analog voltages need to be converted to digital output for further processing. To that effect, various ADC architectures have been adopted in memristive crossbars, including successive approximation register (SAR)-based ADC and flash ADC [93], [94]. While flash ADCs [see Fig. 8(a)] offer high speed, they suffer from high area/complexity and high input capacitance limiting its maximum precision to 6 bits [94]. Typically, flash ADCs employ parallel sampling to achieve fast conversion, and an n -bit flash ADC consists of 2^n comparators, a resistor ladder, and a decoder. Due to a large number of comparators, the energy consumption of a flash ADC is much larger than the same-precision SAR ADC [95].

SAR ADC is the most common ADC architecture adopted in analog in-memory computing memristive crossbars due to its simplicity and low area/energy overhead compared with other ADC architectures [95]. The SAR architecture exploits a binary search algorithm in a feedback loop, including 1-bit ADC (comparator). Fig. 8(b) shows a conceptual schematic of a standard SAR ADC consisting of a comparator, n -bit DAC, and SAR logic. The SAR ADC algorithm starts with setting the SAR MSB value to “1,” and the rest of the bits are zeros, producing $V_{DAC} = V_{REF}/2$. Then, the comparator is enabled to decide whether $V_{sampled}$ is higher or lower than V_{DAC} . The resulting decision is fed to the SAR logic to configure the DAC to produce V_{DAC} equal to either $V_{REF}/4$ or $3V_{REF}/4$. To produce N bits, the SAR loop takes N clock cycles. Various ADC architectures and precisions have been adopted in memristive crossbar MVM units [29], [30], [88]. For instance, ISAAC [29]

and PUMA [30] adopted an 8-bit SAR ADC for higher precision computations, while XNOR-RRAM implemented a 3-bit flash ADC for binary input/weight computation [57]. Note that the ADC area and energy costs are huge and dominate the whole crossbar area and energy [30]. Typically, as reported in [35], ADC consumes 80.61% and 68.13% of the crossbar power and area, respectively. Therefore, ADCs are shared across the columns, and analog multiplexers (MUX) and sample and hold (SnH) blocks are adapted to allow the time-multiplexed ADC usage for further energy efficiency. On the other hand, reducing the ADC precision further reduces its area and energy.

It is worth mentioning that the ADC precision required to compute exact MVM operation in an $N \times N$ crossbar having K input precision and M weight precision should be $\text{ceil}(\log_2((2^K - 1) * (2^M - 1) * N))$. Moreover, the energy consumed by the ADC is a function of its precision [36]. By varying the precision of the ADC, we can introduce functional errors as well as lower the energy consumption of the NVM crossbar primitive. Researchers have proposed techniques to reduce the required ADC precision as an attempt to improve efficiency. As an example, ISAAC proposed to store the weights in its original or flipped form to increase the number of zero sums, hence reducing ADC precision. Moreover, recent works have proposed to adopt lower-than-required ADC precision due to the sparse nature of ML workloads and shown negligible accuracy degradation [57]. To conclude, peripheral design choices, such as input encoding, input/weight precision, ADC architecture, and precision, are key factors to improve energy consumption, area, and performance at the crossbar and system levels.

C. Crossbar Write Operations

The operation of resistive crossbars for memory applications uses the more conventional read–verify–write mechanism for reliable programming of devices. However, due to the serial nature of this update mechanism and high write energy of NVM technologies, the cost of *in situ* training of large-scale DNNs using resistive crossbars becomes significantly high. As a solution, researchers have proposed a parallel updating scheme [101], particularly for ML workloads, which leverages the analog mechanism of programming to mimic the outer product operation commonly used in DNN training algorithms. Fig. 9 shows such a parallel writing scheme, where the row inputs x_i are encoded by the pulsewidth and the column inputs y_j encoded as pulse amplitude. Simultaneous application of pulses at the rows and column produces a multiplicative effect such that a higher voltage at the column and longer pulse at the row results in a bigger change in conductance of the NVM devices at the crosspoint. The result, of course, is the update of the weights, expressed as

$$W'_{ij} = W_{ij} + x_i \times y_j. \quad (3)$$

Table 2 Silicon Demonstrations

Category	[22]	[21]	[96]	[97]	[98]	[57]	[99]	[88]	[100]
Technology Node (nm)	2 μ m	180 nm	180 nm	130 nm	130 nm	65 nm	65 nm	55 nm	22 nm
Device Type	RRAM	PCM	RRAM	RRAM	RRAM	RRAM	RRAM	RRAM	RRAM
Crossbar Size	128x64	512x1024	64x64	128x16	784x100	128x128	256x64	256x512	512x512
Input Precision	4b	NA	1b	1b	1b	1b	1b	1b, 2b	1b, 2b, 4b
Weight Precision	2b	NA	8b	5b	3b	1b	4b	3b	2b, 4b, 4b
MAC output	4b	NA	1b	8b	8b	3b	8b	4b	6b, 10b, 11b
Accuracy (MNIST)	99%	97.95%	90.8%	NA	94.4%	98.43%	NA	98.8%	NA
Accuracy (CIFAR-10)	NA	88.29%	NA	94%	NA	86.08%	NA	88.52%	90.18%
Accuracy (CIFAR-100)	NA	67.96%	NA	NA	NA	NA	NA	NA	66.46%
Accuracy (ImageNet)	NA	NA	NA	NA	NA	NA	69%	NA	NA
Energy Efficiency ¹ (TOP/s/W)	115	28.065 ²	20.7	11	78.4	141.18	5.9	53.17	121.38 (6b) 45.52 (10b) 28.93 (11b)

¹ TOP/s/W values are reported with respect to the corresponding technology node and the output precision.

² scaled to 14 nm.

Weights in crossbar are stored in the unsigned form. To account for both positive and negative weight updates, the inputs are represented in sign-magnitude form. The polarity of weights can be represented using two separate devices for positive and negative weights. To represent both positive and negative weights in the same device, the range of weight, R_{ON} to R_{OFF} , is symmetrically centered around $(R_{ON} + R_{OFF})/2$ [102]. This means that a zero weight is represented by $(R_{ON} + R_{OFF})/2$. Such sign-magnitude and biased representation of weights can enable both positive and negative updates to the same device. The polarity of the update is controlled by the polarity of the simultaneous pulses [103]. Such parallel writing schemes can be adopted entirely in the time domain as well [104]. Such a scheme has also been experimentally demonstrated on floating gate memory arrays [105].

We have discussed the basic anatomy of resistive crossbars, starting from the building blocks (NVM devices at the

crosspoint) to a detailed description of peripheral circuits required for proper operation of the crossbar as compute cores in DNN accelerators. The devices and circuits, however, can introduce undesirable behavior that affects the desired MVM functionality of the crossbars. In the next section, we will study the origin of such nonidealities and their impact on the MVM functionality.

D. Silicon Demonstrations

1) *NVM MVM Macros*: Although NVM technologies are in their nascent stage of developments, there are several experimental demonstrations of MVM macros based on these technologies. There has been extensive work on experimental demonstration of PCM [21], [43] and RRAM crossbars [56], [57], [89]. At the time of writing this article, experimental demonstrations of Spintronic and FeFET crossbars are yet to be explored in a large-scale. Table 2 summarizes the recently demonstrated memristive crossbars configurations and results.

2) *CMOS MVM Macros*: It is worth mentioning that MVM units can also be constructed using CMOS-based SRAM cells [13]–[15], [106]. Fig. 10(a) shows an example of such a macro based on an eight-transistor (8T) SRAM array [14]. The bit cell consists of six-transistor (6T) SRAM cell along with a read port constituted by two additional transistors. The node “Q” stores the weight bit. By applying an appropriate voltage on the read WL (RWL), the current sourced from the SL onto the BL becomes a function of the stored weight bit and the applied voltage. The currents from all the bit cells in a single-column accumulate in the BL, which represent the MVM output, similar to resistive crossbars. To enable multibit MVMs, the transistors in the read port can be appropriate in the ratio 8:4:2:1, as shown in Fig. 10(a). If the weight $W_i^j = w_3w_2w_1w_0$, where w_i are the bits corresponding to the 4-bit weight, the vector-matrix dot product becomes

$$\Sigma(v_i \cdot W_i^j) = \Sigma[v_i \cdot (2^3w_3 + 2^2w_2 + 2^1w_1 + w_0)]. \quad (4)$$

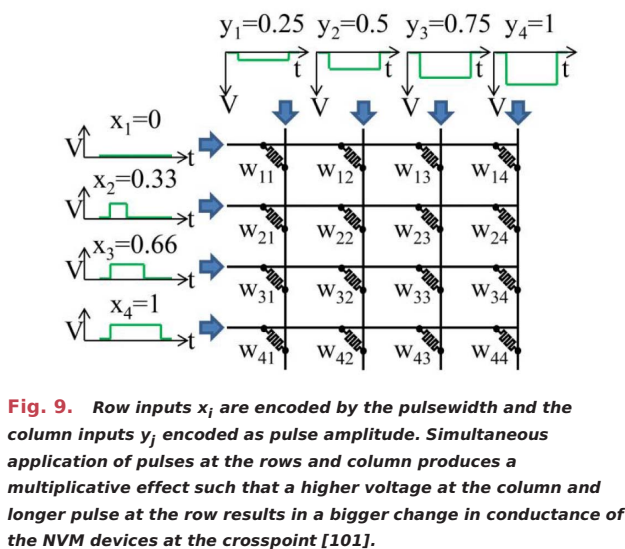


Fig. 9. Row inputs x_i are encoded by the pulsewidth and the column inputs y_j encoded as pulse amplitude. Simultaneous application of pulses at the rows and column produces a multiplicative effect such that a higher voltage at the column and longer pulse at the row results in a bigger change in conductance of the NVM devices at the crosspoint [101].

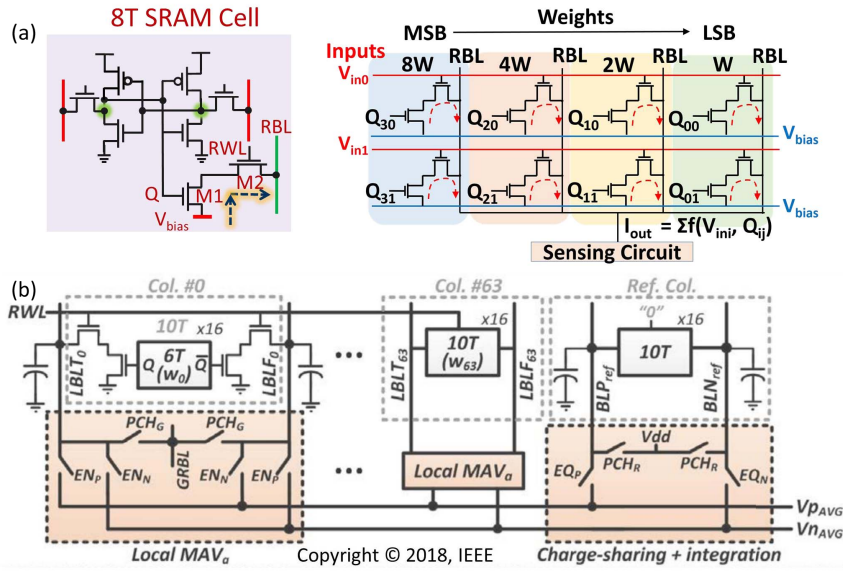


Fig. 10. (a) MVM macro using current-based computations in 8T SRAM cells. The weight bit is represented by node “Q.” By applying a voltage on RWL, the current in the RBL is a function of the weight bit and the applied voltage. The transistors are sized appropriately to represent multibit weights. (b) MVM macro using voltage-based computation in 10T SRAM cells. The local bit-lines (LBLT and LBLF) are discharged according to the weight bit stored in the 6T SRAM. Through charge sharing, the average MVM output is calculated in the horizontal lines [15].

Alternatively, voltage accumulation can also be used to perform MVM operations through charge sharing in the BL. Fig. 10(b) shows a voltage-based MVM operation in a ten transistor SRAM cell [15]. This bit cell consists of two local BLs, which are discharged according to the weight bit stored in the bit cell. The local BLs are appropriately shorted to evaluate the average MVM output, which is then passed to a charge sharing ADC to obtain the digital output. Due to the inherent nonlinearity in transistor characteristics, the linearity of MVM operations in CMOS memories is difficult to realize. Moreover, as SRAM cells can only store single weight bits, CMOS-based analog computing MVM macros lose in area efficiency to resistive crossbars for multibit MVM operations. As a result, researchers have explored CMOS macros primarily in the context of binary neural networks [107], [108] where the matrix–vector multiplications are represented using XNOR operations [57], [109] between binary inputs and weights. Despite several efforts investigating possibilities of MVM computations using CMOS memories, their significantly higher area footprint with respect to resistive crossbars make the latter more promising as basic compute primitives for ML workloads. Throughout the rest of this article, we focus on the design principles of resistive crossbars for ML workloads.

III. CROSSBAR NONIDEALITIES

As discussed earlier, the analog nature of computing in NVM crossbars can lead to approximations in their functionality as MVM units. Such approximations arise from

device and circuit nonidealities originating from metal lines, peripheral circuits, and nonideal devices. While we will study how such approximate analog computing systems can outperform digital counterparts in Section IV, it is important to delineate the sources of these approximations and their impact on the functionality. In this section, we provide a detailed analysis and perspective on the implication of the imperfections in resistive crossbars on MVM computations. Later, in Section V-C, we will delineate several techniques to overcome the effects of these nonidealities. The mentioned device–circuit nonidealities can be categorized into two types: 1) read nonidealities and 2) write nonidealities.

A. Read Nonidealities

Generally, read nonidealities refer to the imperfections in the devices and circuits that can lead to functional read errors in MVM computations. Furthermore, read nonidealities can be categorized into two parts:

- 1) linear read nonidealities;
- 2) nonlinear read nonidealities.

1) *Linear Read Nonidealities:* Linear read nonidealities comprise three kinds of parasitic resistances: 1) wire resistance; 2) source resistance; and 3) sink resistance. Fig. 11 shows a typical memristive crossbar with parasitic resistances.

a) *Wire resistances:* These originate from the interconnects between memristive cells in BLs and WLs. Typically, the wire resistance can be calculated as

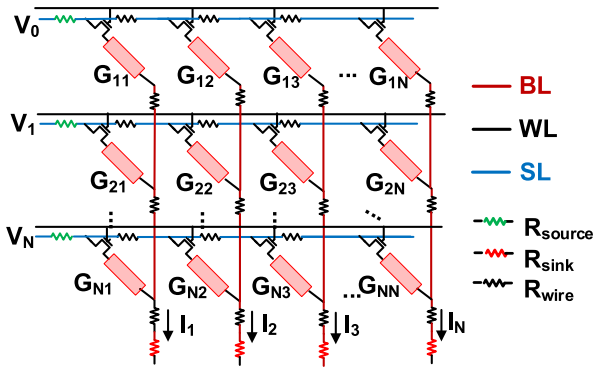


Fig. 11. Resistive crossbars with parasitic resistances, arising from BL and WL wire resistances (R_{wire}), input driver resistance (R_{source}), and sensing resistance (R_{sink}).

$R_{\text{wire}} = (\rho l / wh)$, where ρ , l , w , and h are the interconnect resistivity, wire length, width, and height, respectively [110]. Therefore, the wire resistance parasitic effect increases with crossbar size due to the increase in wire lengths. Furthermore, with technology scaling, the wire resistance increases at more advanced nodes due to the reduced width [110].

b) *Source and sink resistances:* These represent the WL analog drivers output resistance and the input resistance of the current sensing circuitry, respectively. Such parasitic resistances contribute significantly to current–resistance (IR) drops across the BLs and WLs of the crossbar, thus degrading the output current from its ideal value.

2) *Nonlinear Read Nonidealities:* Nonlinear read nonidealities originate from the nonlinear behavior of both NVM devices and access transistors. The read operation is affected by the nonlinear current (I) versus voltage (V) characteristics of the NVM devices. Besides NVM devices, the transistor characteristics can also introduce nonlinearity in the computations.

a) *Device read nonlinearity:* Different device technologies can exhibit different kinds of nonlinear behavior due to the fundamental dissimilarities in current conduction. For example, transport in amorphous PCM devices has been described by the Poole–Frenkel transport [111] of carriers, which results in the current being linear at small voltages and exponential at higher voltages. In the crystalline state, PCM devices exhibit Ohmic behavior at low voltages and non-Ohmic behavior at high voltages due to Joule heating. The compact model for such nonlinear I – V behavior has been previously studied [112], [113].

Unlike PCMs, RRAM conduction relies on tunneling mechanism that results in an exponential dependence of current on voltage. This can be expressed as [59] $I(d, V) = I_0 \exp(d/d_0) \sinh(V/V_0)$. Here, d is the gap size between the tip of the filament and electrode; I_0 , d_0 , and V_0 are fitting parameters.

Similarly, spin devices show voltage-dependent resistive characteristics. In addition, the nonlinear I versus

V characteristics in spin devices can be modeled using nonequilibrium Green’s function (NEGF) [114]. Fig. 12(a) shows the nonlinear I – V characteristics of: 1) PCM; 2) RRAM; and 3) spintronic technologies for $R_{\text{ON}} = 10$ k Ω . It can be observed that all the NVM technologies exhibit I – V characteristics that deviate from the expected linear resistive behavior. Such nonlinear behavior can result in functional errors when MVM units are constructed using these devices as MVM operations demand linearity of the output (current) with the input variables (V and G). Indeed, nonlinearities during the read process in resistive crossbars can pose a challenge toward accurate MVM functionality.

b) *Access device nonlinearity:* The nonlinearity of access devices can play a role in introducing functional inaccuracies in MVM units. Various access devices have been adopted in crossbar-based MVM units, as mentioned in Section II-B1. Such access devices (two-terminal selectors or transistors) impose different nonlinear characteristics on the analog computing operations in crossbars.

i) *Two-terminal selectors:* Two-terminal selectors can switch the current passing into the memory device by exponential switching or threshold switching, as mentioned in Section II-B1. Interestingly, 1ES-1R-based crossbars show more nonlinearity effects over different technologies than 1TS-1R-based crossbars. According to [79], 1S-1R cell I – V characteristics strongly depend on the selector type (ES or TS) after the RRAM state has been changed, which is clearly shown in the linear scale I – V curves in Fig. 12(b). ES inherently shows a slow slope of several hundred mV/decades due to the exponential switching nature. Therefore, a certain amount of the cell voltage is continuously applied to ES even after it is turned on during the 1ES-1R operation, resulting in the nonlinear I – V curves, as shown in Fig. 12(b-i). On the other hand, the applied voltage to 1TS-1R cell beyond TS threshold voltage is fully transferred to the RRAM due to the fast slope that is at most 5 mV/decade [79]. As a result, the linear I – V trace of the 1TS-1R device governed by the RRAM is shown in Fig. 12(b-i).

ii) *Transistors as access devices:* Generally, 1T-1R cells behave more linearly than 1S-1R. The reason for that is transistors in 1T-1R cells operate in the linear region during the MVM operation. Moreover, there is no threshold for the applied voltage to reach before switching since it is a three-terminal switch. To that end, 1T-1R exhibits less significant nonlinearity than two-terminal selector-based cells with the cost of a large cell area.

B. Write Nonidealities

The conductance tuning operation in NVM devices is desired to be linear and symmetric, especially for online weight update algorithms. However, a typical curve between the conductance and the number of programming pulses can significantly deviate from the desired linear and

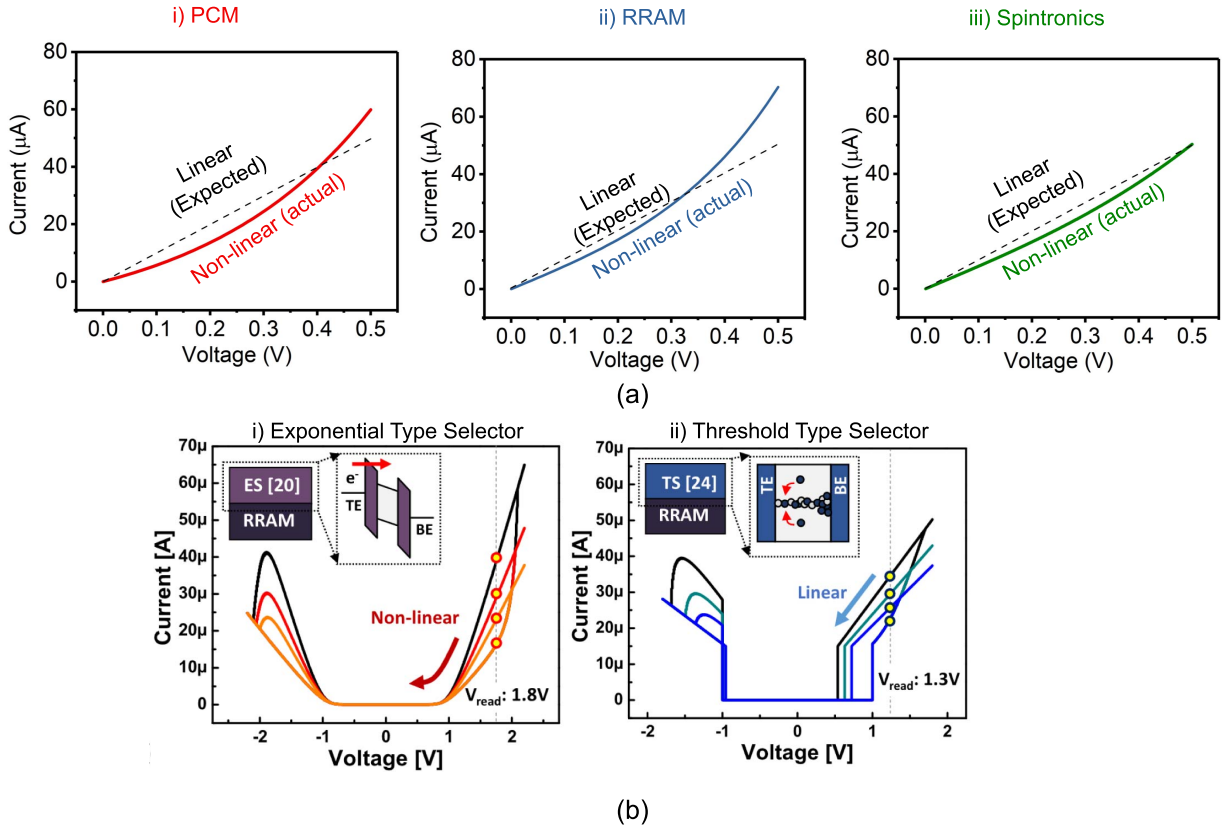


Fig. 12. (a) NVM nonlinearity (I - V characteristics for a typical): (i) PCM; (ii) RRAM; and (iii) spintronic device for $R_{on} = 10\text{ k}$ exhibiting nonlinear behavior, resulting in significant deviation from expected linear characteristic. (b) Access device nonlinearity: (i) I - V trace of the 1ES-1R follows the exponential curve of the ES (exponential-type selector); and (ii) semilinear I - V curve of the 1TS-1R caused by the RRAM [79] (threshold-type selector).

symmetric behavior, as shown in Fig. 13(a). This behavior has been quantitatively modeled by researchers for RRAMs [115], [116] through the following equations:

$$G_+ = B \left(1 - e^{-\frac{P}{\alpha}} \right) + G_{min} \quad (5)$$

$$G_- = -B \left(1 - e^{\frac{P-P_{max}}{\alpha}} \right) + G_{max} \quad (6)$$

$$B = (G_{max} - G_{min}) / \left(1 - e^{-\frac{P_{max}}{\alpha}} \right) \quad (7)$$

where P is the number of applied pulses, G_+ (G_-) is the conductance trajectory in the increasing (decreasing) direction, G_{min} (G_{max}) is the minimum (maximum) conductance, and P_{max} is the maximum pulse number required to entirely switch the device between two extreme states. The parameter α controls the nonlinearity behavior, whereas B is a fitting function. Typically, α can range from $+6$ to -6 , where $\alpha = 0$ denotes the linear behavior. Fig. 13(b) shows the nonlinear conductance trajectory for varying range of α . In this figure, we also observe that due to the asymmetric trajectory of increasing and decreasing conductance, the same difference in number of applied pulses can lead to drastically different conductance updates.

The nonlinearity and asymmetry in NVM devices adversely affect the online learning schemes due to the

continuous and gradual weight update in such schemes. For offline trained neural networks, the conductances of the NVM devices in the crossbar are predetermined and, hence, can be reliably written using the popular read-verify-write mechanism [117].

C. Impact of Nonidealities on the Output Current

The impact of the aforementioned nonidealities arising from device characteristics and circuit parasitics introduces functional errors in MVM computations (read nonidealities) as well as weight updates (write nonidealities) during online training.

1) *Read Nonidealities:* We study the impact of the read nonidealities on the output current of the crossbar. We consider RRAM device characteristics for simplicity. To study the cumulative impact of each nonideality, we consider a 64×64 1T-1R crossbar and perform SPICE simulations for various input V and G matrix combinations. The parameters for SPICE simulations are presented in Table 3. Fig. 14 shows that different voltage (V) and conductance (G) conditions, which leads to similar ideal current (I_{ideal}) that can result in a varying range of nonideal current ($I_{nonideal}$) outputs, causing errors in computations. The difference can be quantified using a nonideality factor (NF) that is

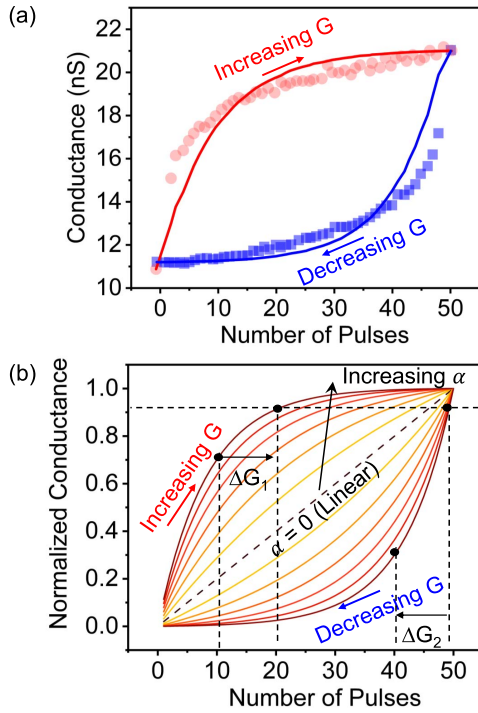


Fig. 13. (a) Conductance (G) evolution curve with respect to number of programming pulses [115]. (b) Nonlinear conductance curve model versus number of programming pulses for varying degree of nonlinearity, showing asymmetry between increasing and decreasing conductance trajectories.

the relative error (R.E) between I_{ideal} and $I_{nonideal}$. NF is calculated as

$$\frac{I_{ideal} - I_{nonideal}}{I_{ideal}} \quad (8)$$

Fig. 14(a) and (b) shows the I_{ideal} versus $I_{nonideal}$ characteristics for the crossbar under the influence of, first, only linear nonidealities (blue) and, second, both linear and nonlinear nonidealities (red) for different supply voltages (V_{supply}). We observe that the effects of nonlinearity become more prominent at higher V_{supply} , resulting in a higher difference in NF between the red and blue plots.

We study the impact of individual contributions of different nonidealities mentioned in Section III-A.

Table 3 Crossbar Simulation Parameters

Category	Parameter	Value Ranges	References
Design Parameters	Crossbar Size	16, 32, 64	Design Choices
	R_{source}	1000 Ω	
Circuit Non-idealities	R_{sink}	150 Ω	
	R_{wire}	2.5 Ω	
Device Parameters	I_0	0.1 mA	[118], [119]
	g_0	0.25 nm	
	V_0	0.25 V	
	Number of bits	1, 2, 4	[19], [21], [23], [40]
	ON Resistance	50k, 100k, 300k	
	ON/OFF Ratio	2, 6, 10	

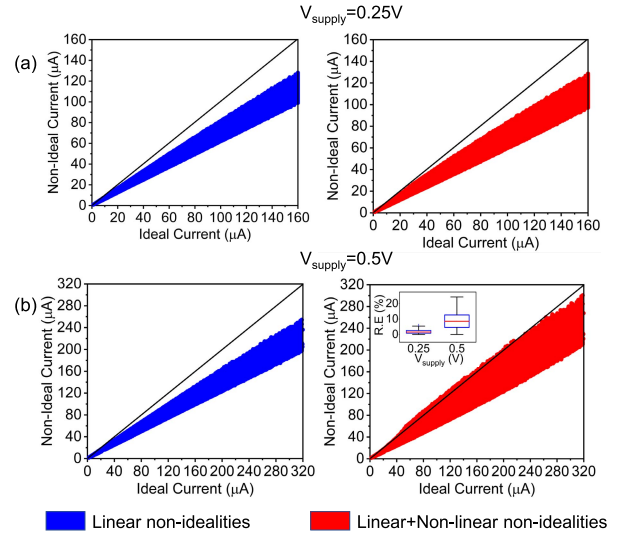


Fig. 14. Ideal (I_{ideal}) versus nonideal ($I_{nonideal}$) current plots for (a) $V_{supply} = 0.25$ V and (b) 0.5 V showing that the case with both linear and nonlinear nonidealities have higher errors than solely linear nonidealities, particularly for higher supply voltages. Inset: R.E for $V_{supply} = 0.25$ V/0.5 V in $I_{nonideal}$ between the two cases (blue and red).

The simulations based on the parameters listed in Table 3 show that source resistance has the highest mean NF among linear nonidealities in Fig. 15. This is due to high source resistance. The nonlinear device and transistor nonidealities have a negative NF, i.e., it causes

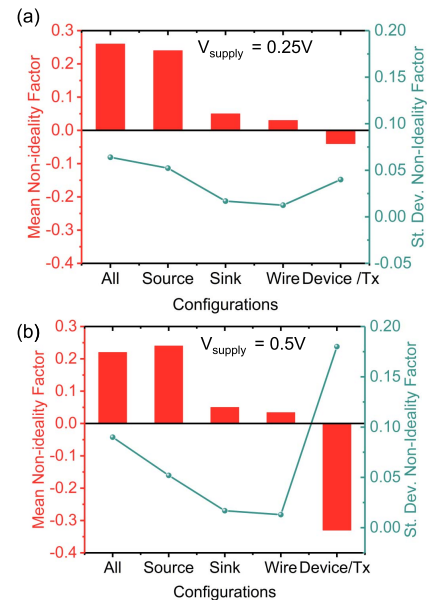


Fig. 15. Mean and standard deviation of NF for (a) $V_{supply} = 0.25$ V, and (b) $V_{supply} = 0.5$ V, highlighting the individual contributions of the different sources of read nonidealities mentioned in Section III-A. The configuration "All" refers to the case when all nonidealities are considered.

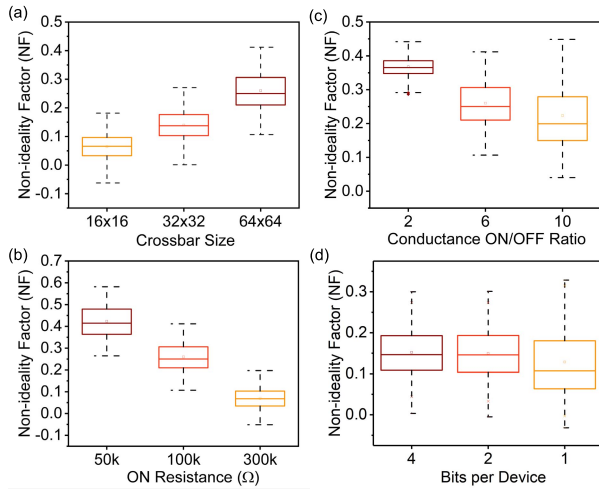


Fig. 16. Impact of read nonidealities on resistive crossbar output, expressed as NF [see (11)], on crossbar design parameters, such as (a) crossbar size, (b) conductance ON/OFF ratio, (c) ON-resistance, and (d) bits per device.

I_{nonideal} to be higher than I_{ideal} , unlike the linear nonidealities. This effect is pronounced for higher supply voltages (0.5 V). The standard deviation of NF is also primarily contributed by device nonlinearity, which gets significantly magnified for higher supply voltages. Thus, the device and transistor nonlinearities act in the opposite direction to the linear nonidealities. This necessitates accurate modeling of these nonlinear nonidealities.

Read nonidealities have a significant impact on crossbar design parameters both from the device and circuit point of view. These design parameters are, namely; 1) crossbar size; 2) ON-resistance; 3) ON/OFF ratio; and 4) bits per device.

a) Crossbar size: The size of the crossbar has a considerable effect on the impact of parasitics on MVM functionality. The reason is twofold. First, larger crossbars consist of long metal lines, thereby increasing the impact of wire resistance. Second, higher crossbar size also amounts to lowering the effective resistance, as seen from the input WL drivers. A lower effective resistance results in the increasing impact of parasitic resistances on the MVM functionality. Fig. 16(a) shows the trend of NF for varying crossbar sizes from 16×16 to 64×64 . It can be observed that mean NF increases by $\sim 5\times$ as crossbar size increases from 16×16 to 64×64 . Although using lower crossbar sizes would reduce functional errors in large-scale DNNs, it would also lead to more crossbars for representing a particular network. This results in energy-latency-accuracy tradeoff from a system design point of view.

b) ON-resistance: The ON-resistance is referred to as the resistance of the LRS in the NVM device. For a given crossbar size and resistance ON/OFF ratio, the effective resistance of the crossbar seen from each input is also determined by the ON-resistance. A lower ON-resistance

reduces the effective resistance, which results in an increased detrimental effect of parasitic resistances. Fig. 16(b) shows that mean NF is $\sim 5\times$ higher for $R_{\text{ON}} = 50 \text{ k}\Omega$ than for $R_{\text{ON}} = 300 \text{ k}\Omega$. It is intuitive to use higher ON-resistance to reduce the impact of parasitics on the MVM functionality. From an energy point of view, using higher ON-resistance can also reduce the array energy due to lower passive power ($P = V/R$) dissipation. One disadvantage of using higher ON-resistance, however, is need for higher sense margin that can lead to higher peripheral overhead.

c) ON/OFF ratio: For a fixed ON-resistance and crossbar size, the ON/OFF ratio of the NVM device resistance also affects the impact of nonidealities on the output current of the crossbar. This is because, for a given ON-resistance, a lower ON/OFF ratio reduces the average resistance in each column of the crossbar. Thus, a lower ON/OFF ratio results in a higher impact of parasitics. Fig. 16(c) shows that mean NF is $\sim 1.75\times$ higher for ON/OFF ratio = 2 than for ON/OFF ratio = 10. However, a ON/OFF ratio = 10 has a higher deviation in NF, implying a strong dependence of NF on the varying resistance range in the crossbar. Despite that variation, a higher ON/OFF resistance ratio is desirable in NVM devices for proper MVM functionality in resistive crossbars.

d) Bits per device: Finally, the bits per device can have some effect on NF, as shown in Fig. 16(d). It can be observed that although there is not a significant difference between the NF distribution when the number of bits per device is 2 or 4, storing 1 bit per device certainly results in a lower mean NF but with higher deviation. This is because storing 1 bit per device skews the resistance distribution in the crossbar toward either R_{ON} or R_{OFF} , thus increasing the effective ON/OFF ratio of the crossbar. Thus, this impact can be correlated with a higher ON/OFF ratio when a similar trend is observed. Packing a lower number of bits per device might reduce the average NF, but it harms the storage density of the crossbars, which will reduce the area efficiency.

The variation in NF with various design parameters provides us with insights on how to design crossbars with lower NF. Lower crossbar sizes with high ON-resistances, high ON/OFF ratio, and low bits per device might seem preferable, but, as we have described before, each of these design choices has their own negative implications. Thus, careful design space exploration needs to be performed to achieve optimal energy-latency-accuracy tradeoff.

2) Write Nonidealities: The impact of write nonidealities becomes prominent during online training on crossbars. To quantify the impact of the write nonidealities, we study the deviation of the conductance matrix under the influence of write nonlinearity and asymmetry explained earlier from the desired values. Increasing degrees of nonlinearity, α , in (8), can affect the iterative conductance update quite severely. Fig. 17 depicts the impact of device write nonlinearity and asymmetry on the final conductance achieved after 20 weight update iterations. The mean

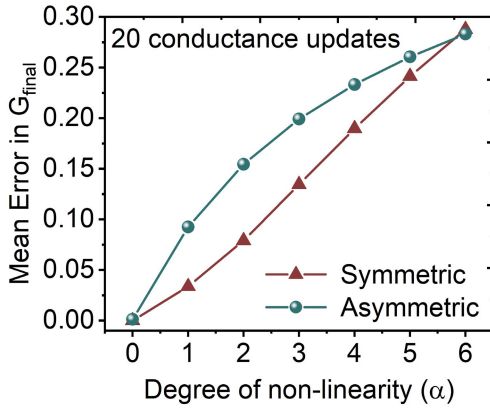


Fig. 17. Impact of device write nonlinearity and asymmetry on conductance on mean error in final conductance, G_{final} , calculated by the mean R.E between the desired (linear) final conductance value after 20 updates and the achieved final conductance. Here, symmetric (asymmetric) update means that the conductance trajectories, as shown in Fig. 13(b), are identical (different) for negative and positive weight updates.

error in conductance G is calculated as the mean R.E between the desired (linear) final conductance value after 20 updates and the achieved final conductance. The symmetric updates exhibit lower error than the asymmetric updates although, for high nonlinearity, both can have a significant deviation from the ideal. In Section V-C, we will delineate the impact of device write nonlinearity and asymmetry on application accuracy for systems with online learning.

D. Process Variations

So far, we have discussed nonideal device and circuit behavior leading to errors in MVM computations and weight update operations. In addition, a major challenge in realizing large-scale resistive crossbars is process variations, which can add undesirable randomness to the device behavior. Broadly, process variations in resistive crossbars can be categorized into the following:

- 1) device-to-device write variations;
- 2) cycle-to-cycle write variations;
- 3) device-to-device read variations.

1) *Device-to-Device Write Variations*: During the write process, NVM devices can exhibit considerable variation from one device to another in the same crossbar. This can be modeled by adding a variation term in the nonlinearity programming equation [see (5)–(7)]. The degree of nonlinearity in programming can vary device to device with a typical $3\sigma = \pm 1.5$. Fig. 18(a) illustrates how device-to-device variations manifest in the form of programming characteristics.

2) *Cycle-to-Cycle Write Variations*: Cycle-to-cycle write variations can be described as the variations in conductance characteristics at every programming pulse.

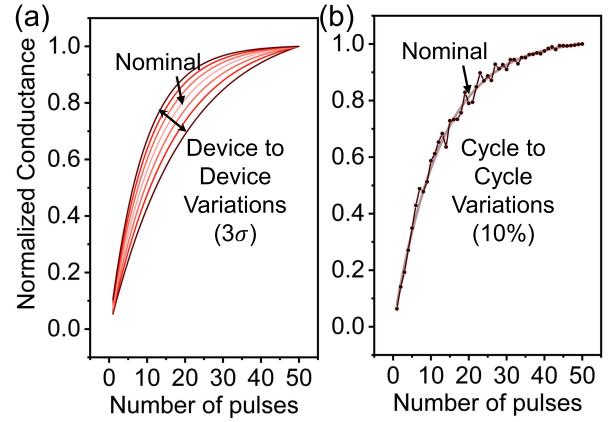


Fig. 18. Impact of (a) device-to-device variations and (b) cycle-to-cycle variations on normalized conductance versus number of pulses characteristics.

It can be modeled by considering distribution in the nonlinearity factor for different programming pulses. Fig. 18(b) illustrates how device-to-device variations manifest in the form of programming characteristics.

The write variations primarily affect the online learning schemes on NVM devices. We will study the impact of such variations on application accuracy in Section V.

3) *Device-to-Device Read Variations*: The device-to-device variations can also reflect in the I – V characteristics of the device, which can affect the MVM functionality of resistive crossbars. To analyze such an effect, the variations can be included in the fitting parameters of the compact model of such devices. For example, in RRAM devices, (4) can be modified to include the effect of variations through the parameters d_0 and V_0 . Fig. 19 illustrates how device-to-device variations manifest in the form of I – V characteristics.

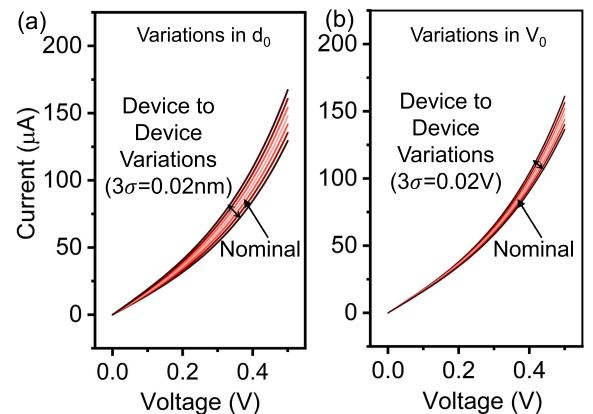


Fig. 19. Device-to-device variations manifesting in the read operation of the NVM device. Variations in the fitting parameters (a) d_0 and (b) V_0 in (7) results in a variation in I – V characteristics.

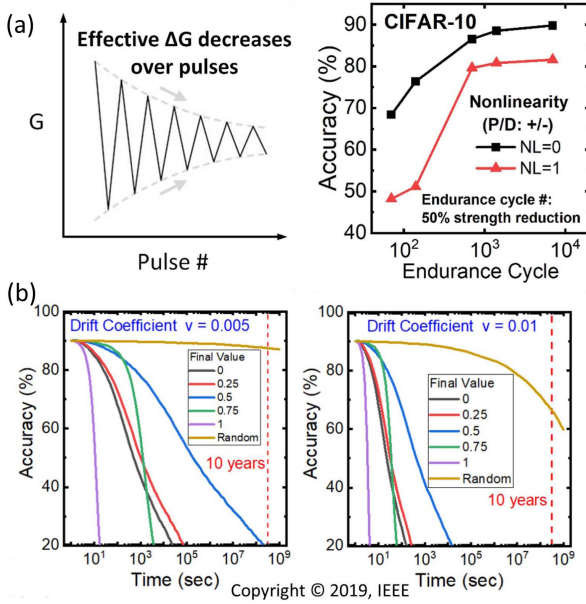


Fig. 20. (a) Conductance strength decaying over number of pulses and its impact on inference accuracy for varying endurance cycle. (b) Inference accuracy versus data retention time showing degrading accuracy for various conductance drifting modes [116].

E. Write Endurance and Data Retention

Typically, NVM devices exhibit undesirable characteristics in memory properties, such as write endurance and data retention. As resistive crossbars are primarily associated with analog weight updates, the strength of updates can degrade with the number of programming pulses applied [120], as shown in Fig. 20(a). This behavior can be modeled as [116]

$$\Delta G = G_0(1 - r)^{N_{\text{PULSE}}} \quad (9)$$

where r is the ratio that controls the strength of update, G_0 is the desired conductance change, and N_{PULSE} is the number of write pulses. The impact of this undesirable characteristic is studied [116] on CIFAR-10 data set where the training accuracy is severely affected, especially under the effects of write nonlinearity, and a good endurance cycle ($\sim 10^4$ cycles) is necessary for proper training.

Next, data retention is the ability of NVM devices to store data for a significant amount of time (typically > ten years). There could be different conductance drifting modes, where the conductance of the device can move toward a maximum, minimum, and intermediate states [121], [122]. In the context of NVM for ML workloads, the drifting effects have been analyzed [116] by a widely used PCM drift model [123]

$$G = G_0 \left(\frac{t}{t_0} \right)^v \quad (10)$$

where G_0 is the initial conductance, v is the drift coefficient, and t_0 is the time constant. Considering different final conductance values, the inference accuracy of CIFAR-10 data set degrades below 20% before the ten-year threshold for each case except considering random drifting, as shown in Fig. 20(b). For a higher drift coefficient, even the impact of random drifting is severe enough to degrade the accuracy to 67% in ten years. The impact of drifting, thus, necessitates periodic refreshing of data for the proper functioning of the implementation. In addition to the aforementioned nonidealities, NVM technologies suffer from other kinds of reliability issues arising from manufacturing defects and faults raising testing challenges [124], [125].

F. Impact of Temperature

Besides inherent device and circuit nonidealities, the ambient temperature has a profound effect on the conductance of the NVM devices [126] and, hence, on the MVM operations. Researchers have studied [89] this impact by modeling the temperature dependence of NVM conductance and, subsequently, benchmarking with experimental data. Fig. 21(a) shows experimental current (I)–voltage (V) characteristics of a TaO_x device showing significant dependence on ambient temperature (varying from 300 K to 450 K). The ambient temperature above 300 K causes an error in the MVM functionality. The mean and standard deviation of the error distribution rise drastically above 360 K.

G. Crossbar Modeling

In this section, so far, we have delineated the different sources of errors that affect the crossbar read (MVM compute) and write operations. In large-scale DNNs, which involve millions of MVM computations, the errors incurred at each MVM unit can accumulate, leading to a significant degradation in application accuracy. In order to mitigate the adverse effects of nonideal crossbar hardware, it is first necessary to analyze and evaluate the impact on application accuracy. One approach to perform this is functionally abstract the nonideal MVM operation and integrate it into

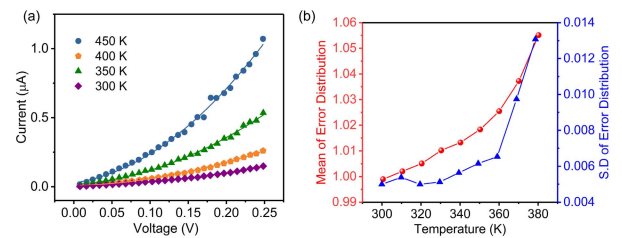


Fig. 21. (a) I - V Characteristics variation under the influence of different ambient temperatures. (b) Mean and standard deviation of error distribution of dot-product operations in NVM crossbars increase with increasing ambient temperature (redrawn from [89]).

existing ML frameworks (such as PyTorch [127], TensorFlow [128], and Caffe [129]). This necessitates modeling the behavior of realistic crossbars under the influence of various nonidealities and variations. Broadly, crossbar modeling techniques can be categorized into two types, as shown in Fig. 22: 1) analytical modeling and 2) statistical modeling.

1) *Analytical Modeling*: The MVM operation on a resistive crossbar may be modeled by expressing it using analytical equations that describe the output currents as functions of input voltages, NVM device conductances, and various other parasitics. However, the direct evaluation of these equations will be too slow to enable meaningful evaluation of large-scale applications. Fast crossbar model (FCM) [130] is a weight-transformation-based approach to enable fast analytical modeling of resistive crossbars. FCM comprises of two modeling phases: 1) model generation phase and 2) model evaluation. In the model generation phase, the weight matrices are first transformed to nonideal weight matrices by abstracting the hardware instance, the synaptic device, and the interconnect specific crossbar nonidealities arising due to process variations, nonlinear synaptic conductance characteristics, sensing resistances, source resistances, and wire resistances. The weight transformation is performed by a crossbar model generator where the input arguments are crossbar parameters and a weight matrix (W) and the output is a nonideal conductance matrix (G_{nonideal}). G_{nonideal} is a modified effective matrix to account for the nonidealities by transforming the original matrix G_{ideal} such that $V \times G_{\text{nonideal}}$ represents the nonideal output current. The crossbar model generator performs linear algebraic operations (roughly corresponding to solving a system of linear equations) to compute G_{nonideal} . In the model evaluation phase, FCM emulates the core resistive crossbar array using the precomputed G_{nonideal} matrix. The nonlinear ADC and DAC models are called as preprocessing and postprocessing functions. By representing the core crossbar array as matrix multiplication, thereby enabling the use of optimized basic linear algebra subprograms (BLASs) routines, this modeling technique can achieve five orders of magnitude higher speed than HSPICE with good accuracy. As we will describe in Section V, FCM can effectively model the impact of a variety of crossbar nonidealities, and the weight transformation technique can be seamlessly integrated with ML frameworks to study the impact of crossbar-based computation on large-scale DNNs.

An alternative method to model resistance nonidealities was proposed by Jeong et al. [131], where the WL and BL resistances were approximated by lumping them into one large resistance in series with the device resistance. This assumption is valid only when the wire resistances are significantly lower than the device resistances. Using such lumped models, the approximate output current can be obtained by adding the lumped WL and BL resistance

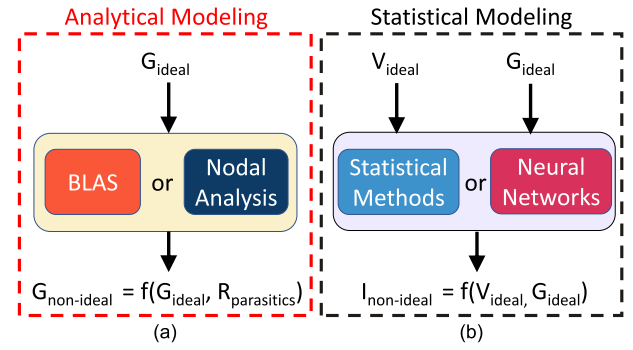


Fig. 22. Crossbar modeling techniques can be broadly categorized into two types. (a) Analytical modeling. (b) Statistical modeling. Analytical modeling techniques model the crossbar including its nonidealities by deriving and evaluating analytical equations for the outputs. Statistical modeling techniques apply statistical methods such as regression or neural networks to model the functionality of the crossbar.

to the device resistance. The impact of parasitics can be also modeled using modified nodal analysis, as adopted in [131].

2) *Statistical Modeling*: Analytical modeling can effectively capture the impact of linear and nonlinear nonidealities when the input variables, namely, V and G are mutually independent. That is, the nonideal current I_{nonideal} can be expressed as a function of uncorrelated variables, V and G . However, the voltage-dependent nonlinear device characteristics arising from the NVM devices and access transistors can lead to an interdependence of V and G , especially in the presence of parasitics. This is because parasitics modify the voltage of the nodes in the crossbar, and the voltage-dependent characteristics of the NVM devices result in a codependent behavior of voltage and output. This requires iterative solvers, such as HSPICE, to provide accurate outputs that are difficult to integrate with large-scale DNN frameworks. To address this issue, data-dependent modeling is required to emulate the generalized behavior of the nonideal crossbar.

Data-dependent models could be based on statistical models, such as regression, which can effectively model the output behavior of a resistive crossbar, based on input V_{ideal} and conductance matrix G_{ideal} . Such a regression model can be obtained using a 3-D data fitting of nonideal output current I_{nonideal} as $I_{\text{nonideal}} = f(V_{\text{ideal}}, G_{\text{ideal}})$. Such a regression model can be extended to a multilayer perceptron (MLP) network to generalize the behavior of the resistive crossbar [132]. The ability of the MLPs to project data to a high-dimensional space enables them to predict the output based on different input patterns. The input neurons of the MLP is a concatenation of V_{ideal} vector and a flattened G_{ideal} vector. The output of the MLP represents I_{nonideal} as a function of V_{ideal} and G_{ideal} . In order to train the MLPs, there is a need to create a data set of V_{ideal} and G_{ideal} that effectively span over

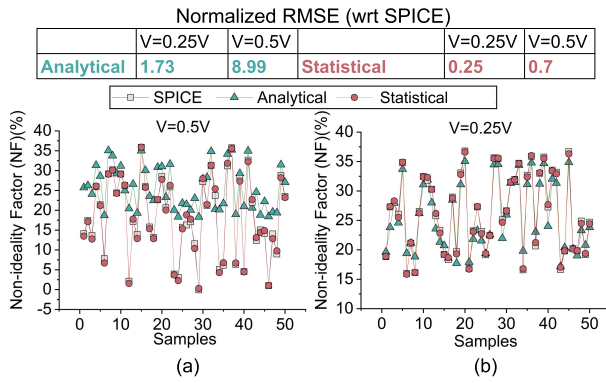


Fig. 23. Benchmarking of crossbar models with HSPICE.

(a) Analytical modeling can effectively capture linear nonidealities.

(b) Statistical modeling performs significantly better for capturing linear and nonlinear nonidealities.

the input space of the resistive crossbars in large-scale DNN frameworks. Data distribution in DNNs is inherently sparse. As a result, such a data set can be formed by exhaustively considering various degrees of sparsity in the input V_{ideal} and G_{ideal} data. Data-dependent models can effectively capture both linear and nonlinear nonidealities described in Section II-C. In comparison to analytical models that only capture the linear nonidealities, an MLP based data-dependent model can achieve $\sim 7\times$ and $\sim 11\times$ lower root-mean-square error (RMSE) in NF (as described in Section II-C) with respect to the HSPICE data for $V_{supply} = 0.25V$ and $0.5V$, as shown in Fig. 23. Modeling of nonideal crossbars is a crucial step toward designing large-scale crossbar-based hardware. Such models should be able to seamlessly integrate with existing ML frameworks, such as PyTorch, Tensorflow, or Caffe, to help analyze and mitigate the effects of crossbar nonidealities on application-level accuracy. We discuss such software frameworks for resistive crossbar-based systems in Section V. However, we first discuss how the building blocks discussed in Section II can be used to design large-scale crossbar-based hardware architectures for DNN inference and training.

IV. ARCHITECTURE

Typical ML workloads have large model sizes (hundreds of million parameters) and are dominated by MVM operations. Furthermore, depending on the application, an inference workload (i.e., batch size of 1) can have low data reuse. For instance, convolutional networks exhibit both weight and input data reuse, but recurrent networks have weight data reuse only, and MLPs have no data reuse [30]. Since CMOS-based architectures have limited on-die storage (tens of MBs in GPU) due to the low storage density of SRAMs, ML models are typically stored using off-chip memory, such as DRAM. As shown in Table 4, the energy cost associated with DRAM access is orders of magnitude higher than the energy cost of SRAM access and computation. Consequently, the execution of ML workloads on

CMOS-based systems is dominated by off-chip memory accesses. To this effect, the in-memory computing benefits of resistive crossbars can be leveraged to design ML accelerators for overcoming the off-chip memory bottlenecks.

The key features of a resistive crossbar that drive the design of a crossbar-based accelerator are: 1) high storage density and 2) high write cost (energy and latency). First, the compact cell structure ($4F^2$ cell area for the in-line two-terminal selector configuration) [103] and the multibit storage device (2–6 bits) [89] enable resistive crossbars to achieve order-of-magnitude higher on-chip storage density compared with SRAM ($124F^2$ [134]). Large on-chip memory can enable efficient data reuse to optimize inference efficiency. Second, the high write voltage [103] and multiple programming cycles (program-verify approach [45]) result in much higher write cost (energy and latency) than SRAM. Expensive memory writes limit the applicability of a time-multiplexed architecture, where the crossbars are reused across layers by reprogramming weight matrices and executing the corresponding MVM operations. Consequently, a spatial architecture where the DNN is partitioned such that the weights (stationary data) are pinned to crossbars located across multiple cores is more efficient as it leverages the benefits of high storage density while alleviating the costly writes.

A. ReRam-Based Inference Accelerators

In an inference workload, crossbars typically perform MVM operations with stationary matrices, i.e., DNN weights. Thus, the spatial architecture can be mapped with DNN weights during configuration (writing weights to crossbars) and can be subsequently used for millions of inferences, thereby amortizing the cost of configuration. In addition to the MVM operations performed by crossbars, DNNs require several other operations, such as vector additions, vector multiplications, scalar, and nonlinear operations. Furthermore, other nonstationary data, such as inputs and partial sums, need to be stored and moved. Consequently, designing a computation core requires augmenting the resistive crossbar with CMOS-based digital logic units as well as memory units to provide the required generality for DNN workloads. A naive approach toward integrating the CMOS and NVM technologies is not viable because of the huge disparity in compute and storage density between the two technologies. CMOS digital logic (MAC) has an order of magnitude higher area requirement than a crossbar of equal output width ($\sim 20\times$) [30].

Table 4 Energy Cost of Different Hardware Components for 45-nm CMOS Process [133]

Operation	Energy (pJ)	Relative cost
32-bit int ADD	0.1	1
32-bit float ADD	0.9	9
32-bit Register File	1	10
32-bit int MULT	3.1	31
32-bit float MULT	3.7	37
32-bit SRAM Cache	5	50
32-bit DRAM Memory	640	6400

Moreover, a crossbar's storage density (2-bit cells) can be 160 MB/mm^2 , which is at least an order of magnitude higher than SRAM (6T, 1-bit cell) [29].

Resistive crossbars have been used to build special-purpose ML accelerators, such as SPINDLE [28], RENO [135], PRIME [136], ISAAC [29] for convolutional neural networks (CNNs), and MLPs. Past research has also proposed using memristive crossbars for spiking neural networks [137], [138] and the Boltzmann machines [139]. These efforts have focused on specific types of neural network applications (one or two) and have achieved substantial improvements compared with CMOS-based general-purpose architectures (CPU and GPU) and ASICs. Going beyond, it is imperative to expand the scope of the workloads and explore if a crossbar-based system can be extended to a general-purpose ML accelerator. The application-specific accelerators that execute limited types of neural networks typically use a state-machine-based approach where a layer's functionality is encoded (hardwired) as state machines. Such a hardwired approach can lead to increased decoding overhead and complexity, particularly when the scope of workloads is large. Consequently, there is a need to think in terms of instruction set architecture (ISA) programmability. Furthermore, enabling flexible data movement and control operations to capture the variety of access and reuse patterns in different DNNs is important. Since crossbars have high write latency, they typically store constant data, while variable inputs are routed between them in spatial architecture. This data movement can amount to significant energy consumption, which calls for flexible operations to optimize the data movement. PUMA [30], a recently proposed inference accelerator, addresses these needs by considering a general-purpose and ISA-programmable accelerator built with resistive crossbars.

B. Microarchitecture of a Typical ReRAM-Based Inference Accelerator

Typical ReRAM-based accelerators are spatial and organized in hierarchies: core, multicore, and node [29], [30], [136]. Fig. 24 shows the logical organization of a core, which consists of an instruction execution pipeline, digital CMOS-based functional unit, and analog crossbar-based MVMU. The instruction execution pipeline and specialized ISA enable representing multiple ML workloads using a small set of custom instructions. Such programmability is important to ensure that an accelerator can cater to the ever-increasing variety of ML workloads. The choice of custom instructions provides a tradeoff between efficiency and accelerator's generality. The functional unit is based on single-instruction-multiple-data (SIMD) and executes linear and nonlinear vector operations. ML workloads typically have high data parallelism and execute wide vector operations, which motivates having wide vector instructions. On the other hand, hardware considerations motivate having narrow vector width to avoid offsetting the

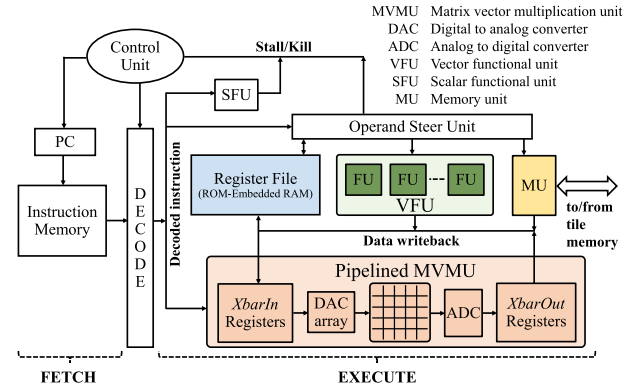


Fig. 24. Core architecture based on hybrid CMOS-memristive technology. The VFU is four-lane wide. The MVMU comprises eight 128×128 crossbars with each cell representing 2 bits. The crossbar receives inputs from 1-bit DAC array and sinks its output to a 8-bit ADC, shared across all 128 columns [30].

area efficiency of crossbars. Therefore, a time-multiplexed narrow SIMD unit balances the tradeoff between workloads favoring wide vector width and hardware favoring narrow vector width. The choice of SIMD width provides a tradeoff between vector instruction's efficiency and MVM efficiency. The MVMU executes dot-product operations in the analog domain in crossbars. In addition to the crossbars, an MVMU consists of DAC and ADC to interface with the digital data since all other computations and communication occur in the digital domain. Typical MVMUs use bit-slicing to represent high precision data (input and weights) required in ML [29]. Here, input bits are streamed using low-resolution DACs, and weight bits are partitioned across low-precision NVM devices. DAC reuse across crossbars that share the inputs and ADC reuse across a crossbar's columns reduce the area overhead from peripherals to enable an efficient MVMU design. The choice of crossbar size, DAC resolution, NVM-device's precision, and ADC resolution provide control knobs to vary the energy efficiency and area efficiency of MVMU.

Fig. 25 illustrates a multicore that consists of several cores connected together with shared memory. The

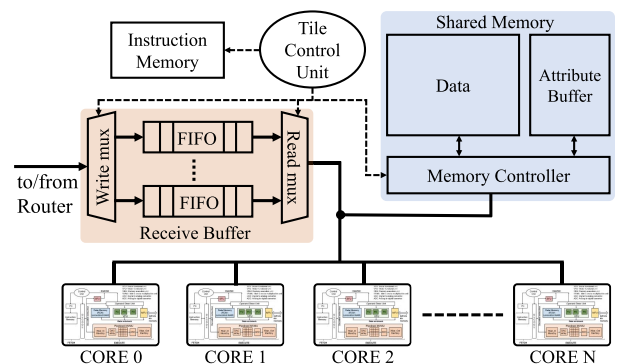


Fig. 25. Multicore architecture [30].

shared memory enables data movement between cores both within a multicore as well as between different multicores. Subsequently, the choice of shared memory size and optimizations to reduce the communication between cores reduces the data movement cost. It is worth noting that the data movement in a spatial architecture can be seen as analogous to the conventional data movement within the memory hierarchy in temporal architectures (CPU and GPU), where the cost increases from intracore communication to intercore communication. Multiple multicores are connected together using an on-chip network to form a node.

C. Energy Benefits of ReRam-Based Inference

Fig. 26 shows the inference energy of a ReRam-based accelerator [30] compared with a CPU (Intel Skylake) and a GPU (NVIDIA Pascal). Note that Pascal obtains lower energy and latency than Skylake for all benchmarks. In addition, across all benchmarks, the ReRam-based accelerator achieves significant energy reductions at lower or comparable latency than Pascal. CNNs show the least energy reductions (up to 13 $\times$) due to the abundant weight and input reuse, which is leveraged well by CMOS systems (CPU and GPU) to amortize the expensive memory accesses. MLPs show up to 80.1 $\times$ energy reductions, while LSTMs show the highest energy reductions of up to 2446 $\times$. While both LSTMs and MLPs have little to no weight reuse, MLPs show lower savings than LSTMs due to their smaller model sizes. A small model (few million parameters) that can be fit on the on-chip memory (last-level cache) on CPUs or GPUs does not expose the high off-chip memory energy consumption.

Compared with the Google TPU (CMOS ASIC), the ReRam-based accelerator demonstrates peak area and power efficiency improvements of more than 9.7 $\times$ and 1.8 $\times$, respectively, across all ML workloads. It is worth noting that the peak efficiency of ReRam-based systems does not get affected by data-batching. However, TPU's efficiency is almost an order lower for lower data-batching (batch-size = 1) due to its inability to amortize the weight data movement.

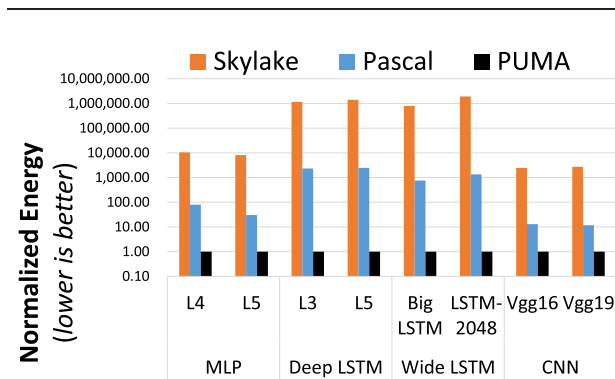


Fig. 26. Inference energy comparison with respect to CPU (Skylake) and GPU (Pascal) for batch size 1 [30].

D. Training Accelerators

Training workloads pose additional requirements for designing crossbar-based systems, namely: 1) larger shared memory and 2) frequent crossbar writes. During inference, the computed activations can be discarded once they are fed to the following layers. However, during training, these computed activations need to be stored, as they are also required during the weight update operations for computing the gradients. Therefore, a system for DNN training has much higher storage requirements for nonstationary data (i.e., shared memory) than a system for DNN inference. For example, a relatively simple ResNet-50 DNN model operating on the ImageNet data set generates ~ 50 million activations during the forward pass, resulting in a 100 MB (2 B/activation) storage requirement per image. The storage requirement further increases for larger networks, bigger data sets, higher resolution inputs, and concurrent processing of multiple images due to pipeline parallelism. For even larger DNNs, the storage requirements can be multiple gigabytes. Thus, the on-chip memory of crossbar-based inference accelerators is often insufficient, and these accelerators have to be accompanied with high bandwidth and high capacity external memory to enable a training system that is capable of handling large-scale DNNs. Consequently, a training system also requires the traditional memory hierarchy (with multiple levels), to efficiently hide the main memory latency. Second, while crossbar writes are infrequent in inference (configuration-time only), the crossbar is written at the end of every step/batch during the training flow. For example, a typical ResNet-50 DNN model training on the ImageNet data set with batch size 64 and 100 epochs requires 18 million crossbar writes. It is worth noting that while the architectural solutions, such as larger shared memory and memory hierarchy, can provision the memory requirements for training, the expensive crossbar write is a fundamental impediment toward building crossbar-based training accelerators.

E. Overcoming Expensive Crossbar Writes for Training

Recent research has proposed a parallel write technique, as discussed in Section II-C, to overcome the fundamental bottleneck of writing into NVM crossbars. Through this approach, weight gradient computations and updates can be performed *in situ* in resistive crossbars [140]. While such an *in situ* and parallel weight update scheme can mitigate the requirement of expensive crossbar writes in ML training, they are limited to low-precision operations. To that effect, researchers have explored bit-slicing to incorporate parallel write to enable high-precision weight updates required in ML [102].

Although bit-slicing technique is common in mapping weight matrices onto NVM crossbars, supporting weight updates can pose several challenges. During the forward pass, the inputs applied to the rows and the columns

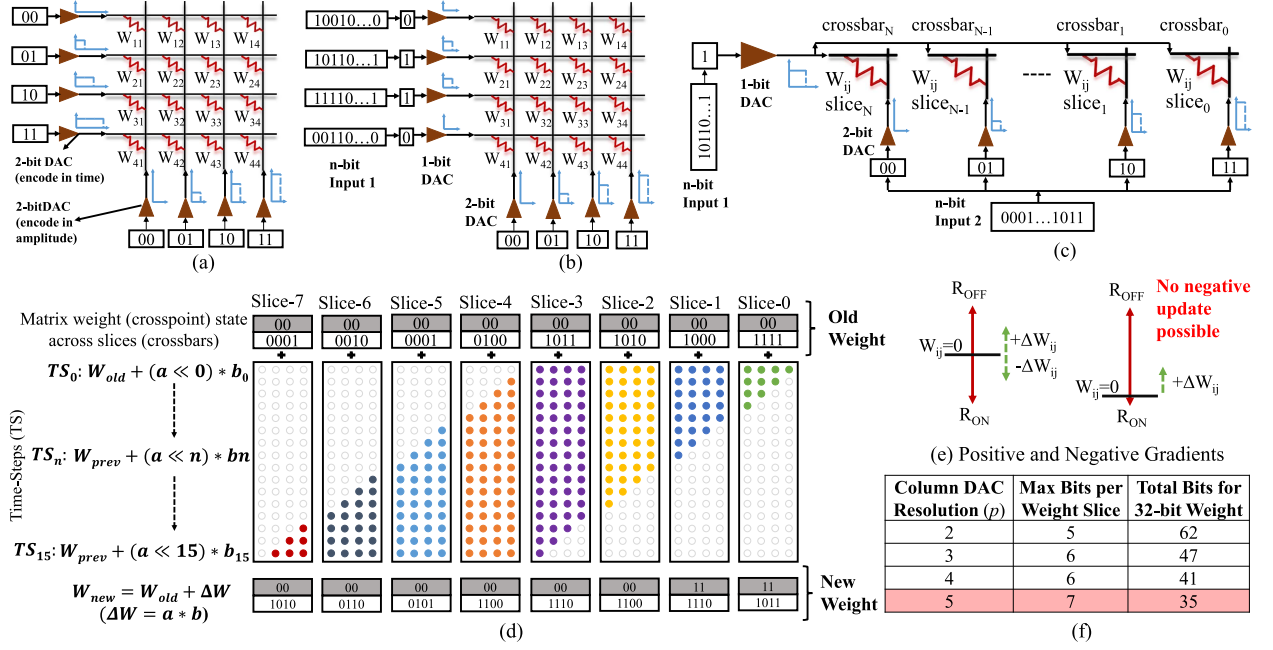


Fig. 27. Bit-slicing weight update to enhance its precision. (a) 2-bit OPA. (b) Streaming row input bits. (c) Slicing column input bits. (d) 16-bit OPA execution. (e) Positive and Negative Gradients. (f) Bit usage for different resolutions.

of the crossbar carry the outputs. In the case of weight update using parallel write, inputs are simultaneously applied to both rows and columns, and the outputs are the resulting weight updates in the crossbar cells. Thus, careful consideration of bit-slicing is required. Moreover, bit-slicing for weight update operations presents additional constraints to the choice of bit distribution across slices. To address such challenges, we describe a recently proposed technique to incorporate bit-slicing in the weight update operation [102] (see Section IV-E1). We further discuss the constraints posed by this technique to the choice of bit distribution and explore ways to address them (see Sections IV-E2–IV-E4).

1) Bit-Slicing the Weight Update Operation: Fig. 27(a) illustrates how the weight update operation is performed when 2-bit inputs are applied at the rows and the columns. The digital row input is encoded in the time-domain using pulsewidth modulation. The digital column input is encoded in the amplitude-domain using pulse-amplitude modulation. Both pulsewidth and pulse-amplitude modulations can be implemented using DACs. The weight change in a cell depends on the duration and the amplitude of the pulses applied on the corresponding row and column, respectively, thereby realizing an analog weight update operation [103], [141].

To perform a weight update operation with 16-bit inputs, naively increasing the DAC resolution is infeasible because DAC power consumption grows rapidly with resolution (N) as

$$P_{DAC} = \beta(2^N/N + 1)V^2 f_{clk} \quad [142]. \quad (11)$$

Instead, we propose an architectural scheme to realize a 16-bit weight update operation by bit-streaming the row input bits, bit-slicing the column input bits, and bit-slicing the matrix weights across multiple crossbars.

Fig. 27(b) illustrates how we stream row input bits, m bits at a time over $16/m$ cycles. Meanwhile, column input bits are left-shifted by m -bits every cycle. Since the number of cycles decreases linearly with m , while the cycle duration increases exponentially with m due to pulsewidth modulation of row input, we choose $m = 1$ to minimize total latency. Using $m = 1$ also means that the row DACs are just inverters, thereby having low power consumption.

Fig. 27(c) shows how we slice column input bits across crossbars. Only one weight W_{ij} is shown for clarity. In each cycle, the left-shifted column input is divided into chunks of p bits ($p = 2$ in this example), and each chunk is applied to the corresponding crossbar.

Fig. 27(d) illustrates the steps for a 16-bit $\times$ 16-bit weight update operation at one crosspoint in the crossbar, resulting in 32-bit output value for each matrix weight. It puts together the bit-streaming of the row input vector b and bit-slicing of the column input vector a with $p = 4$. Each dot represents a partial product ($a_n \cdot b_n$), and the color corresponds to a specific weight slice (crossbar). Thus, the net accumulation of a slice is the result of all partial products of the specific color. The updated weight after a time step T_n can be expressed as

$$W_{updated} = W_{old} + \sum_{n=0}^n (a \ll n) * b_n. \quad (12)$$

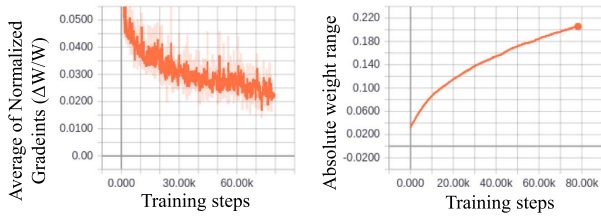


Fig. 28. Weight gradients across training steps.

Crossbars store data in an unsigned form. To enable positive and negative weight updates (δW), we represent inputs in the signed magnitude representation. To enable a symmetric representation of positive and negative weight updates, we bias each device such that a zero weight (W_{ij}) is represented by the memory state $(R_{\text{ON}} + R_{\text{OFF}})/2$, as shown in Fig. 27(e). Hence, the signed magnitude computation and biased data representation enable both positive and negative updates to weights. This is important as both polarities of updates are equally important in DNN training.

2) *Bits to Handle Overflow*: For MVM, the matrix weights are inputs to the operation, and they do not change. In contrast, for weight update, the matrix weights are accumulated with the values resulting from the outer product. As a result, the weight slice stored in a crossbar cell may overflow, either from multiple accumulations within one weight update or over multiple OPAs. We handle this overflow by provisioning weight slices with additional bits to store the carry [shaded bits shown in Fig. 27(d)].

Propagating carry bits to other slices would require serial reads and writes that incur high overhead. For this reason, we do not propagate the carry bits immediately. Instead, they are kept in the slice and participate in future MVM and weight update operations on the crossbar.

The carry bits cannot be kept in the weight slice indefinitely because, eventually, the weight slice may get saturated, i.e., crossbar cell at maximum/minimum state for positive/negative update. Saturation is detrimental to trainability (desirable loss reduction during training) because it freezes training progress due to the absence of weight change. For this reason, we employ a periodic carry resolution step (CRS) that executes infrequently to perform carry propagation using serial reads and writes.

3) *Number of Slices Versus Bits Per Slice*: When slicing matrix bits across multiple crossbars, there is a tradeoff between the number of slices and the number of bits per cell in each slice. MVM operations favor using more slices and fewer bits per slice. The reason is that energy increases linearly with the number of crossbars and nonlinearly with the precision of a crossbar due to the increase in ADC precision required supporting it. Therefore, using more slices with fewer bits each is better for energy consumption.

In contrast, weight update favors having fewer slices with more bits per slice. The reason is that weight update introduces carry bits to each slice and having more slices with fewer bits each increases the overhead from the carry bits. For example, Fig. 27(f) shows that with 2 bits per slice, and 62 total bits are required to represent the 32-bit weight while capturing the carry bits adequately.

To strike a balance, we choose $p = 4$ since $p > 4$ requires a device precision that exceeds ReRAM technology limits [89]. A 4-bit DAC resolution is feasible because DAC power does not increase rapidly at a low resolution [see 11]. By choosing $p = 4$, our MVM operations consume more energy than other ReRAM-based accelerators. However, our more energy-efficient weight update operations compensate because they avoid the need for expensive serial reads and writes.

4) *Heterogeneous Weight Slicing*: MVM operations favor homogeneous bit-slicing. Increasing the precision of any one slice, while decreasing the precision of another, is always an unfavorable tradeoff because energy increases nonlinearly with the precision of a crossbar weight. In contrast, for weight update operations where crossbar values change, provisioning more bits for slices that experience more weight updates helps reduce the frequency of saturation, thereby ensuring trainability at low CRS frequency.

Heterogeneous weight slicing provisions more bits for matrix slices that change more frequently. The frequency of change is impacted by two factors: weight update asymmetry and the small weight gradient range in DNNs. Weight update asymmetry is illustrated in Fig. 27(d), where the central slices receive more partial products (dots) than the edge slices, which motivates increasing precision for the central slices. Small weight gradient range is shown in Fig. 28, where weight updates form a very small fraction (2%–5%) of the overall weight range for $\geq 95\%$ of training steps, which motivates increasing precision of the lower slices.

F. Energy Benefits of ReRam-Based Training

The incorporation of parallel write in ReRam-based training accelerator shows significant energy and performance improvements compared with typical crossbar writes [102]. Fig. 29 shows the training energy of a ReRam-based accelerator that incorporates the bit-slicing technique for training (weight update) compared with a GPU (NVIDIA Turing 2080Ti) for different batch sizes. It can be seen that the ReRam-based accelerator achieves significant energy reductions for both CNN and MLP for all batch sizes. CNNs show lower energy reductions due to the abundant weight and input reuse that is leveraged well by CMOS systems (GPU) to amortize the expensive memory accesses. Furthermore, higher batch sizes show lower reductions as increasing the batch size improves the weight reuse (data-batching).

It is worth noting that ReRam-based training acceleration enables efficient training for a wide spectrum of batch

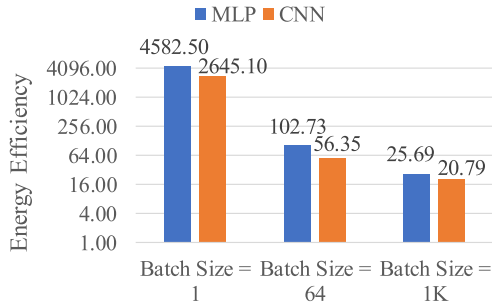


Fig. 29. Training energy comparison with respect to GPU (turing) [102].

sizes (small to large). Training based on small batch sizes is common in several emerging applications, such as lifelong learning [143] and online reinforcement learning [144], where training does not rely on any earlier collected data set.

G. Challenges in ReRam-Based Accelerators

Despite their energy-benefits, further improvements in ReRam-based accelerators are impeded by costly ADCs required in MVMUs. Typical MVMUs used in ML accelerators use 8–10-bit ADCs to guarantee correct dot-product operation, i.e., no degradation from quantization during the analog-to-digital conversion. Such high-resolution ADCs consume significantly higher energy and area compared with the resistive crossbar, thereby dominating the energy and area distribution of an MVMU (as shown in Fig. 30). Recent research has explored microarchitecture optimization for reducing the ADC overhead by dynamically scaling the ADC precision requirements [145]. Here, the lower bits of input and weight data use lower ADC precision, whereas the higher bits of input and weight data use high ADC precision for their crossbar operations. The relationship between the ADC energy versus the effective number of bits (ENOB) represented by the ADC has also been explored by researchers [36]. Further research on approximate computing to reduce ADC precision in certain layers of the ML model, domain-specific ADC designs for ML, and ADC designs at scaled technology nodes will be instrumental toward improving ReRam-based accelerators. In addition to overcoming the impact of peripheral overheads, further research on enhancing the reliability of parallel write for large-sized crossbars can enable the adoption of ReRam-based training accelerators.

V. ALGORITHMS AND TOOLS

Crossbar architectures and data mapping algorithms are crucial in building frameworks to emulate large-scale DNNs using resistive crossbars. Such frameworks are instrumental in studying the implications of resistive crossbar models on application accuracy as well as devising

mitigation techniques to address these shortcomings. Fig. 31 shows a broad summary of Section V.

A. Mapping of DNNs on Resistive Crossbars

One key feature of DNN evaluation frameworks is seamless integration with existing ML frameworks, such as Caffe, PyTorch, or TensorFlow. To that effect, frameworks, such as Ares [146] and Distiller [147], have been developed in congruity with TensorFlow and PyTorch to enable hardware–software codesign. However, these frameworks are based on CMOS systems, which are fundamentally different from resistive crossbar systems. Matrix operations for CMOS systems are modeled using general matrix–matrix multiplications (GEMMs) that use floating- or fixed-point compute units. On the other hand, matrix operations are modeled as MVMs that use resistive crossbars as bit-serial compute units [29], [30]. Similarly, for emulating DNN training, resistive crossbars are used as transposed MVM units with the feature of parallel or serial weight update. In this section, we discuss various approaches toward developing a functional simulator for DNN inference and training using resistive crossbars.

1) *Inference*: The first step toward using existing ML frameworks for large-scale DNN inference evaluation on resistive crossbars is mapping convolutional (conv2d) and fully connected (linear) layers based on MVM computations in crossbars. Note that the impact of memory and communication is ignored in most functional simulators. When the precision of weights in the DNN matches the maximum number of bits that can fit in an NVM device, the weights can be directly mapped using individual resistive crossbars. One such simulator, CxDNN [130], is a functional simulator that maps low-precision DNNs to resistive crossbars. Such a functional simulator can be obtained by modifying the existing ML frameworks to model convolution and inner product DNN layers as matrix–matrix and vector–matrix multiplication operations. It takes a crossbar-based system description, crossbar parameters, and a trained DNN model as inputs and evaluates DNN accuracy using seamlessly integrated crossbar models (explained in Section II-E). Consequently, it abstracts the

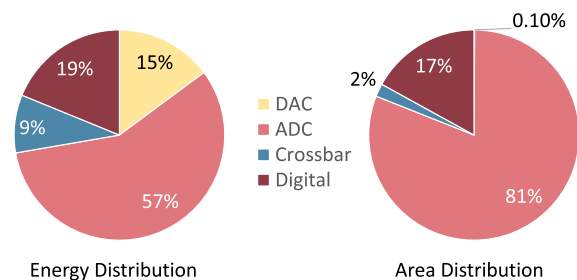


Fig. 30. Energy and area distribution of ReRam-based MVMU.

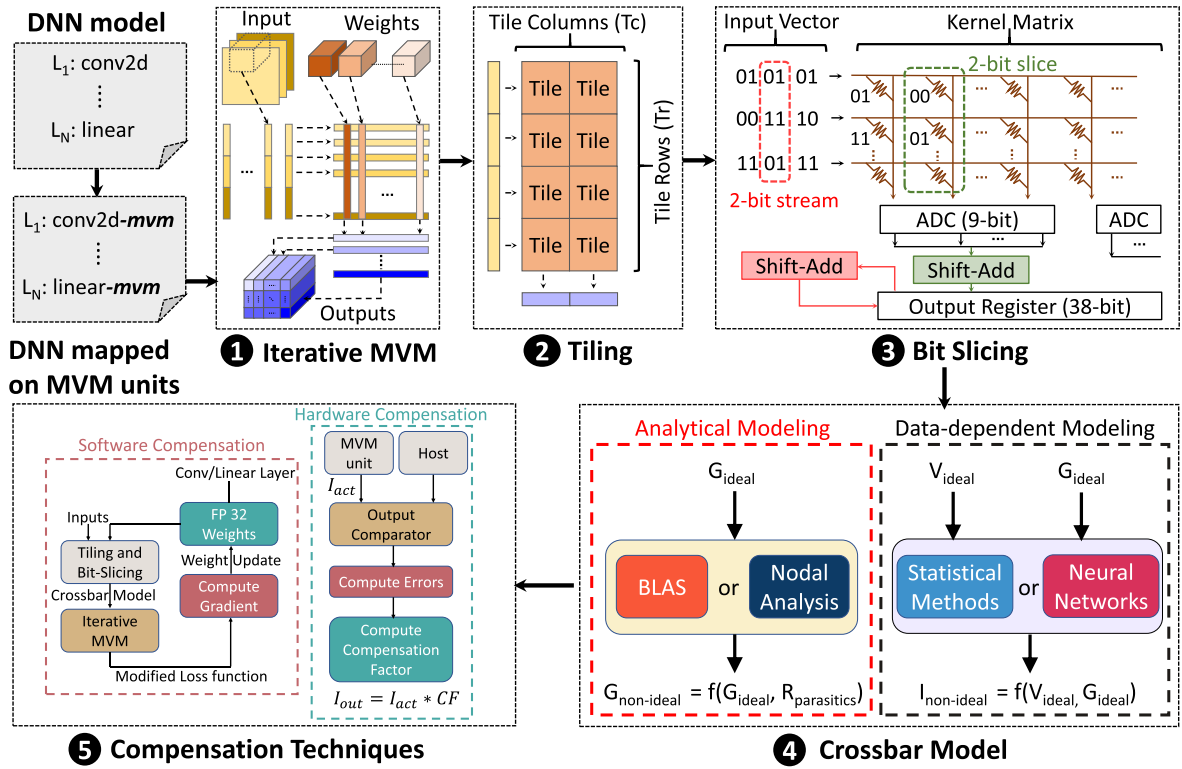


Fig. 31. Large-scale DNN framework on resistive crossbars: 1—iterative MVM expresses convolution operation as repeated matrix–vector multiplication; 2—tiling divides the matrices into crossbars; 3—bit-slicing enable bit-serial computation; 4—crossbar models enable realistic MVM functionality using the device and circuit nonidealities; and 5—framework is used to design analyze large-scale DNNs and design compensation techniques for resulting accuracy degradation.

impact of device and circuit nonidealities into a single programming phase. There are three major steps in the development of a functional simulator. In step 1, it maps DNN to the crossbar-based system provided by the user. The weights are read from the pretrained model and programmed into the crossbar arrays by considering the device-level nonidealities. Subsequently, the conductance matrices (G) corresponding to each crossbar instance are generated. Each G matrix is then transformed to G_{nonideal} matrix using the embedded crossbar models. In step 2, G_{nonideal} matrices are inserted back to the existing ML framework. Finally, in step 3, DNN inference is emulated on crossbars for a set of test inputs using G_{nonideal} matrices and peripheral (ADC and DAC) models. The peripheral models are invoked as preprocessing and postprocessing steps on inputs and outputs of each convolution and inner product layers.

In order to evaluate DNNs with higher precision weights and activations, however, it is crucial to model architectural aspects, such as tiling and bit-slicing, as described in Section IV. Moreover, in order to emulate striding in convolutional layers, each tile is computed across the input image in an iterative fashion. Thus, the emulation of a DNN layer on resistive crossbars consists of three distinct phases; 1) iterative-MVM; 2) tiling; and

3) bit-slicing. First, iterative MVM operations (iterative-MVM) are used to express the convolution operation as repeated MVM operations. Assuming an image classification task, the matrix is formed by the weights, and the vector is formed by a block of pixels across all inputs channels. The output of this operation comprises one pixel from all output channels. Next, tiling is used to divide the weight matrix into several smaller matrices (or tiles). Here, each tile corresponds to the crossbar size. As the tiles constitute the weight matrix, a slice of the input vector is shared across all the tiles in the same row. The tiles in the same column produce partial sum outputs that are added to obtain the convolution output. Finally, bit-slicing of the weights and bit-streaming of the inputs enable bit-serial computations in the crossbar. Within each iteration, input streams are applied to the crossbar to produce outputs from the ADC. The ADC outputs from different weight slices are passed to the shift-and-add units to produce outputs. Finally, the outputs of all the streams go through shift-and-add units to produce the partial sums for a particular tile. Each tile can be represented by a crossbar model, which can model its nonideal characteristics. This functional simulator framework can map large-scale DNNs of any precision onto resistive crossbars by appropriately modeling key architectural aspects, such as tiling and

bit-slicing. Such a framework can accurately analyze the impact of real resistive crossbars with device and circuit nonidealities on DNN accuracy.

2) *Training*: While the aforementioned DNN inference frameworks effectively emulate DNN inference on resistive crossbars with off-line trained weights, on-chip training on nonideal resistive crossbars can be evaluated using DNN training frameworks. Here, we would describe the key aspects of building such evaluation platforms to analyze DNN training on resistive crossbars.

Like DNN inference, DNN training frameworks also need to modify existing ML platforms, such as PyTorch or TensorFlow, to evaluate DNN training and subsequent inference on hardware accelerator architectures, including 1T-1R-based analog resistive crossbars. Such a simulator that incorporates online training on resistive crossbars into ML frameworks is NeuroSim [148]. The mapping of DNNs on such a framework is similar to that in DNN inference frameworks described previously. In addition, it has to also model device write characteristics for the weight update mechanism. At neural network run time, feedforward and backpropagation are iteratively performed through a series of MVM and weight update operations, respectively. Whenever an MVM or a weight update instruction is given, the instruction is passed to the resistive crossbar and device behavioral models for estimating the computation errors.

In addition to undesirable write characteristics in devices, training frameworks can also include circuit-level errors arising from parasitic resistances and peripheral nonlinearities. This can be performed through a three-stage vector-matrix multiplication model to capture the impact of various crossbar nonidealities during both the forward and backward propagation operations. In the first stage, the digital inputs are converted to voltages considering DAC nonlinearity and crossbar nonidealities. In the second stage, the nonidealities within the core crossbar array (interconnect parasitics, sneak paths, and process variations) are modeled as a series of linear-algebraic transformations, wherein ideal conductance matrices are converted to nonideal conductance matrices. Subsequently, in the final stage, digital outputs are obtained from computed currents by considering the ADC nonidealities. A key benefit of the three-stage vector-matrix multiplication model makes it suitable for implementation in GPUs by seamlessly utilizing highly optimized BLAS routines present in standard ML frameworks (e.g., PyTorch, Tensorflow, and Caffe). The weight update nonidealities (stochastic noise [140] and update nonlinearity [116]) are also modeled using optimized BLAS routines.

B. Modeling Tools

Crossbar modeling tools can be broadly categorized into: 1) functional modeling and 2) performance modeling. Functional modeling tools can be used to

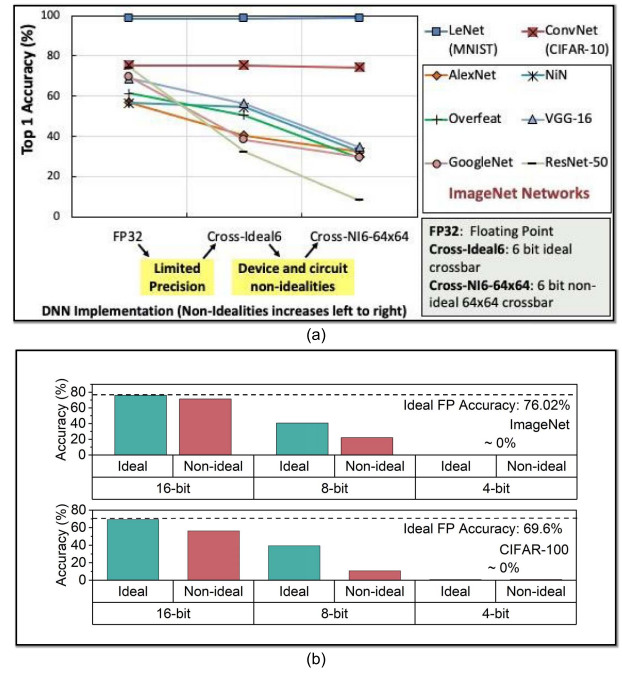


Fig. 32. (a) Impact of linear nonidealities on low-precision networks using CxDNN showing significant degradation in accuracies due to nonidealities (Cross-NI6-64 × 64) with respect to ideal crossbars (Cross-Ideal6), for complex data sets, such as ImageNet (reprinted from [149]). (b) Impact of both linear and nonlinear nonidealities on high-precision networks using features, such as bit-slicing and bit-streaming. It can be observed that for both ImageNet and CIFAR-100, nonidealities have a more detrimental effect on lower precision networks.

evaluate the impact of nonidealities on DNN accuracy. On the other hand, performance modeling tools are used to estimate the energy and latency metrics of crossbar architectures.

1) *Functional Modeling: Accuracy*: Here, we will describe the impact of both read and write nonidealities in crossbars on both inference and training of DNNs.

a) *Inference*: We have previously described the impact of the nonidealities (see Section II-C) on the read operation of resistive crossbars. These errors at the array level can accumulate across layers of a DNN to degrade the application accuracy. First, the impact of limited precision (6-bit) and various device- and circuit-level nonidealities on inference accuracy have been analyzed for simple and complex DNNs. Fig. 32(a) shows the degradation in accuracy for various benchmark DNNs. In this figure, FP32 is a floating-point implementation, Cross-Ideal6 is an “ideal” crossbar implementation that only considers limited precision (6 bits), and Cross-NI6-64 × 64 is an implementation with 64 × 64 crossbars that consider all crossbar nonidealities. We can observe that the accuracy degradation is minimal for small networks (LeNet and ConvNet) on simple data sets (MNIST and ConvNet). However, the degradation in accuracy is substantial (19.8%–57.8%)

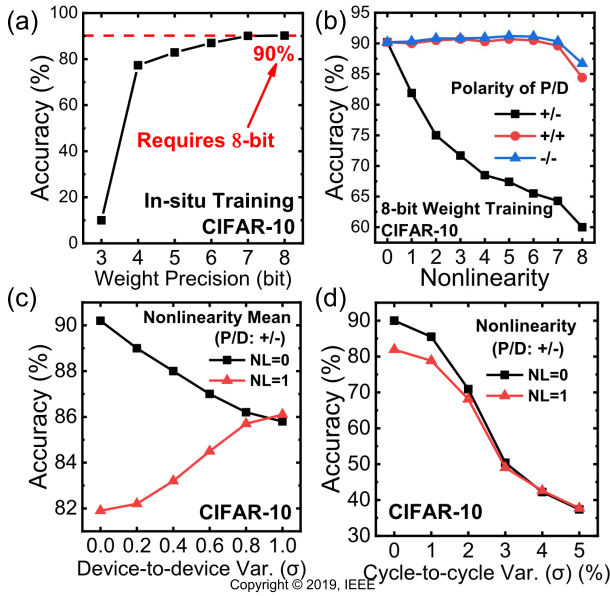


Fig. 33. (a) Impact of weight precision on in situ training and (b) impact of nonlinearity and asymmetry on application accuracy on the CIFAR-10 data set for training of DNNs on resistive crossbars. The impact of (c) device-to-device variations and (d) cycle-to-cycle variations on application accuracy on the CIFAR-10 data set (reprinted from [116]).

for large-scale DNNs on complex data sets, e.g., VGG-16, GoogleNet, and ResNet-50 networks on ImageNet data set, indicating a need for cross-layer error mitigation and compensation techniques to enable low-precision DNNs on resistive crossbar-based systems.

For high precision networks, the impact can be studied using frameworks that include features, such as bit-slicing and bit-streaming, as explained in Section V-A. Fig. 32 shows the accuracy plots with varying bit precision for image classification data sets, ImageNet (subset of 7680 images) and CIFAR-100, for ResNet-18 and ResNet-20 architectures, respectively. In this case, we observe that networks trained with full precision weights and activations have significant degradation when converted to fixed-point representations for 8- and 4-bit precisions. For 16-bit precision, the fixed-point representation can achieve full-precision accuracies. The nonidealities degrade the accuracy by $\sim 4\%$ and $\sim 13\%$ for ImageNet and CIFAR-100, respectively. However, for 8-bit precision, the degradation due to nonidealities is higher ($\sim 18\%$ and $\sim 29\%$ for ImageNet and CIFAR-100, respectively). For 4-bit precision, the accuracies reach 0% for both data sets.

The abovementioned results show that both linear and nonlinear read nonidealities can have a significant impact on DNN inference accuracy. However, undesirable device characteristics can also affect DNN training, which we will discuss in the following.

b) Training: We have previously discussed how to write or weight update mechanisms in resistive crossbars can also pose device level challenges in Section II-C. The impact of the device programming imperfections on application accuracy has been studied using the training framework, NeuroSim, [116] on image classification data sets, such as MNIST and CIFAR-10. First, the precision of the weights has a significant effect on the application as the authors observe that for 8-bit inputs, at least 8-bit precision of weights, are necessary, as shown in Fig. 33(a). Note that for larger data sets, such as ImageNet, precision requirements can be much higher [150]. Given an 8-bit weight training for MNIST/CIFAR-10, the asymmetry in the conductance (G) trajectory versus the number of pulses, as shown in Fig. 17, can remarkably degrade the accuracy as the degree of nonlinearity α increases beyond 1. For symmetric conductance trajectory, i.e., when the programming pulses have the same polarity for both increasing and decreasing conductance, the effect of α seems minimal, as observed in Fig. 33(b). Fig. 33(c) and (d) shows the effect of process variations, such as device-to-device variations and cycle-to-cycle variations on training. Device-to-device variations show an interesting trend where although, for zero α , the accuracy degrades for $\alpha = 1$, the accuracy improves due to the adaptability of neural networks to rely on the device with more symmetric characteristics.

In the presence of the abovementioned device nonidealities, other kinds of linear read nonidealities can also impact the training performance of DNNs on crossbars. The DNN training efficacy can be analyzed under the impact of both device and circuit nonidealities with limited device precision. Fig. 34(a) shows the test accuracy evolution with the number of epochs for nonideal crossbars in comparison with software floating-point and fixed-point representations. The accuracy degradation in nonideal crossbar (Cross-NI) is 4.5%–8.2% across the benchmarks. Furthermore, deeper networks, such as AlexNet DNN, suffer a more detrimental effect than the shallower networks, such as LeNet-5 DNN. Fig. 34(b) shows the sensitivity of test accuracy to crossbar design parameters, such as crossbar size and R_{ON}/R_{OFF} (R_{min}/R_{max} here) ratio. There is a slight drop in accuracy for larger crossbar sizes. However, the lower R_{ON}/R_{OFF} ratios can significantly degrade accuracy when DNNs are trained on the hardware. Although a higher R_{ON}/R_{OFF} ratio is desirable to attain software equivalent accuracies, using arbitrarily high R_{OFF} can result in low absolute sensing currents, adding to errors. This implies a fine tradeoff between sense margin and absolute sensing currents, which necessitates careful crossbar design for facilitating accurate training of DNNs on NVM hardware.

Device- and circuit-level nonidealities, thus, have a significant impact on both DNN inference and training. The read characteristics of devices and other circuit nonidealities pose limitations on crossbar design parameters. The impact of device programming characteristics on training

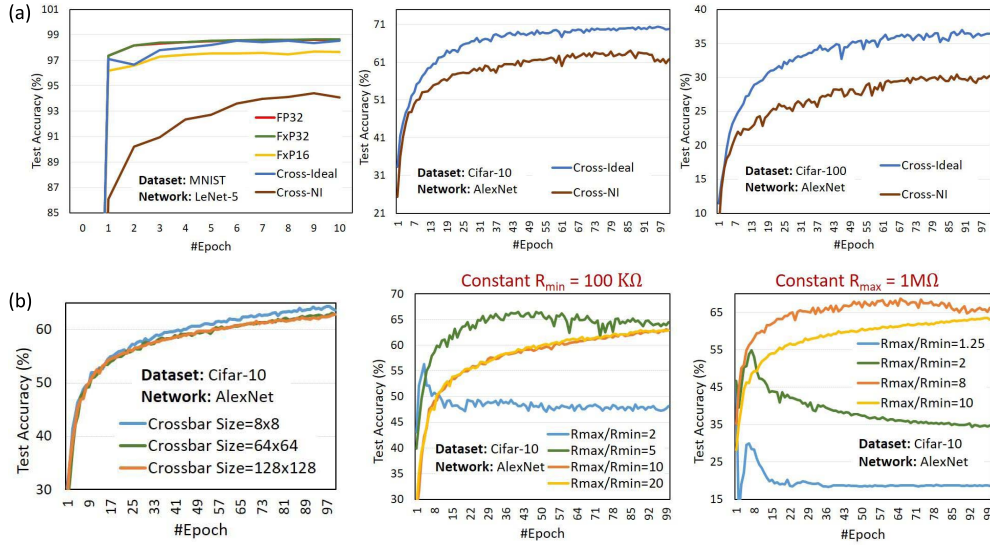


Fig. 34. (a) Test accuracy versus epoch plots for various models, such as LeNet-5 and AlexNet, showing a more detrimental effect of device and circuit nonidealities for the deeper network (AlexNet). (b) Sensitivity of test accuracy to crossbar design parameters, such as crossbar size and R_{ON}/R_{OFF} ratio (reprinted from [151]).

is the constraints on reliability margins of the devices in terms of how much nonlinearity, asymmetry, or variations of the application can tolerate. To operate the resistive crossbars for large-scale DNNs, researchers have explored compensation techniques to recover the accuracy degradation for both inference and training.

2) *Performance Modeling: Energy and Latency:* The energy and latency metrics of NVM crossbar-based architectures can be estimated through performance modeling tools [30], [31]. Previously, in Section IV, we have discussed architecture modeling for calculating energy and latency estimates. Device- and circuit-level modelings of the compute core have been explored using a crossbar simulator, CrossSim, [152] to provide accurate energy estimates for both inference and training operations in DNNs. Such a crossbar simulator can model realistic resistive crossbars, including noisy read and write operations and peripheral nonidealities to provide both accuracy and energy estimates for large-scale DNNs.

C. Compensation Techniques

Over the last few years, researchers have proposed mitigation and compensation techniques to address the device- and circuit-level challenges in resistive crossbars and improve the performance of DNN inference and training. For inference, crossbar models integrated with functional frameworks can be used to retrain the weights of the DNN using gradient descent. On the other hand, compensation techniques for training need to account for nonideal write effects for NVM device programming described in Section II-C.

1) *Inference:* Compensation techniques for inference can be of three types: 1) retraining of weights for error compensation [33], [34]; 2) mapping algorithms to reduce the impact of nonidealities [153]–[156]; and 3) hardware compensation [33].

a) *Compensation through retraining:* When large-scale DNNs are mapped to nonideal crossbars using the aforementioned functional frameworks, it generates modified loss values from ideal DNNs. Such modified loss values can be used to retrain the weights using iterative gradient descent. Such a technique has proven to be effective to recover the accuracy degradation due to the mapping of full-precision DNNs to limited-precision crossbars, as performed by the CxDNN framework, as described earlier. This has been illustrated in Fig. 35(a), wherein accuracy improvement of 8%–27% has been achieved for DNN models, such as AlexNet, VGG-16, and GoogleNet DNNs, with as low as 150 retraining iterations [33]. Such retraining techniques can be further improved by modifying the gradient descent algorithm [34] to account for nonidealities.

b) *Compensation through mapping:* An alternative approach is to use mapping algorithms [89] for weight to conductance conversion by accounting for computational errors arising from device and circuit effects. This approach is also suitable for recovering the impact of defects and variations in the crossbar. For example, a defect map or variation distribution [153], [155], [157] of devices in the crossbar can aid the training process by mapping the sensitive cells to defect-free or low variation cells. Using such defect aware training, the application accuracy has been improved by 50%–80%

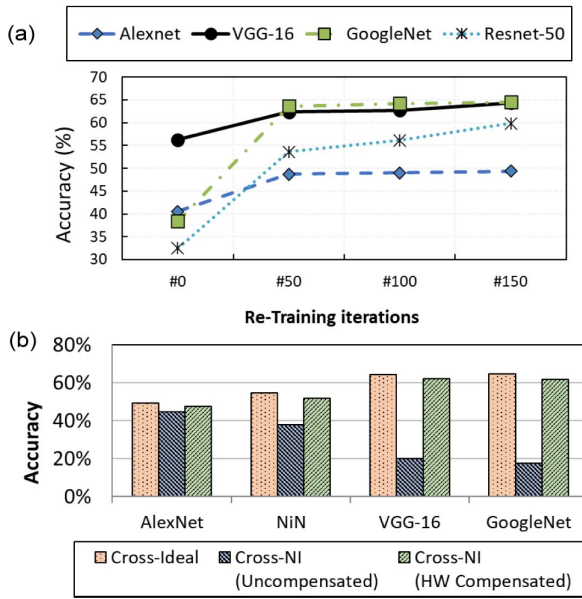


Fig. 35. (a) Accuracy versus retraining iterations plot for various neural network models shows that the degraded accuracy can be recovered using retraining. (b) Hardware compensated networks (Cross-NI) can achieve accuracies close to ideal accuracies (Cross-Ideal).

on the MNIST data set [153]. Other schemes of efficient mapping of weights can also reduce the IR-drops in metal lines. One such technique [154] performs linear algebraic methods to reduce the crossbar size by removing redundant elements to minimize the impact of wire resistances. Wire resistances can also have a larger impact on columns farther away from the voltage source. This can be addressed by mapping the sensitive weights in columns close to the voltage source [156].

c) *Hardware compensation*: The errors in the resistive crossbars can also be compensated by a hardware technique [33]. The outputs of the resistive crossbars, in this case, are multiplied by a compensation factor (CF) to produce close to ideal outputs. CF is calculated by measuring the R.E between ideal output current (I_{ideal}) and nonideal output current ($I_{nonideal}$) for each crossbar used to map the DNN. The hardware compensation methodology can be divided into two steps: 1) design time calibration phase and 2) runtime mitigation phase. In the calibration phase, the host processor to the resistive crossbar accelerator computes CFs based on the R.E between output obtained from the processor (ideal) and output obtained from resistive crossbars (nonideal) for a characterization data set. The computed CFs are then locally stored in the compensation logic of each crossbar. In the mitigation phase, the compensation logic produces CFs for each column, and the output current computed from the crossbar is expressed as $I_{nonideal} * CF$. Such a hardware compensation scheme can achieve accuracies within 1.8%–2.5% of the ideal accuracy, as shown in Fig. 35(b).

2) *Training*: Training a DNN on the hardware can potentially address the functional errors arising from the read nonidealities and fixed-point arithmetic. The inherent adaptability of the gradient descent algorithm can be leveraged in this aspect. This is the reason why retraining DNNs using software models of crossbars is the most popular technique to recover accuracy degradation during inference. However, although such a technique has proven to account for the device- and circuit-level nonidealities to produce a more noise-resilient DNN, hardware shortcoming cannot be exhaustively mitigated using these methods.

a) *Algorithmic Compensation*: The limited ON/OFF ratio in devices and dynamic range of inputs and outputs require algorithmic techniques to circumvent. This is achieved through normalization of the activations and scaling the limited weight ranges appropriately [158]. Another problem of DNN training on hardware is the large difference between the forward pass inputs and backward pass errors, which increases with layer depth. To address this problem, researchers have proposed noise management techniques, such as normalization of inputs and rescaling of outputs [159].

The compensation techniques to circumvent erroneous computations in analog computing primitives have proven to be very effective. It indicates that analog computing system design is strongly tied to the application, and algorithmic modifications need to be performed to truly realize the potential of resistive crossbars as a suitable candidate for such systems. Some of these algorithmic compensations may require digital logic for on-chip compensation. This shows that resistive crossbar-based approximate computing systems need rigorous hardware–software codesign to be viable as future AI hardware.

VI. OUTLOOK

NVM crossbars are promising candidates for accelerating ML workloads with the potential of achieving orders of magnitude higher area and energy efficiency over traditional CMOS computing systems. The area efficiency arises from the ability of NVM devices of storing multiple bits within a cell area of $4F^2$ ($12F^2$ considering 1T-1R cells [160]) compared with $124F^2$ occupied by six transistor SRAM cells [134]. Such high storage densities, coupled with their ability of in-memory MVM operations, enable dataflow architectures with spatially distributed NVM crossbars. The weights in most neural networks can be entirely mapped onto these spatial architectures, thus eliminating the need for costly DRAM accesses that plagues the CMOS computing systems currently. The inherent parallelism within the MVM units as well as their spatial arrangement leads to significantly lower energy and latency consumption than state-of-the-art CMOS digital systems.

Despite significant developments in the domain of NVM crossbar-based computing, there are several challenges at

various levels of design that need to be addressed. We have described how device- and circuit-level nonidealities can impact the MVM functionality of NVM crossbars, severely affecting the application accuracy. Although accurate crossbar models and scrupulous functional frameworks can help us build compensation techniques to mitigate the detrimental effects, NVM crossbars face challenges of data reliability and endurance. PCM and RRAM technologies suffer from limited endurance issues (10^7 cycles) [161]. Moreover, resistance drifting phenomenon [162] in PCM technology can affect data retention, which can have a detrimental effect on NVM crossbars when used as MVM units. Data retention can severely affect neural network inference operations as the weights can vary over time. Limited endurance, on the other hand, can affect neural network training due to a limited number of reliable write operations. These challenges imply that significant material research is necessary to make NVM technology viable for ML workloads.

In addition to data retention and endurance issues, the impact of device variability still remains a major challenge. The dynamic nature of these variations can produce unpredictable and undesirable MVM outputs that may vary remarkably within the crossbars and across the crossbars. Under the effect of device variations, the conductance ON/OFF ratio can pose a limitation on the number of rows that can be simultaneously activated, hence limiting the crossbar size. A high ON/OFF ratio is, thus, desirable for enabling larger crossbars so that the MVM computations have enough sense margins for appropriate sensing through the ADCs. Another implication of device variations is that they induce noise into the MVM computations, thus reducing the signal-to-noise ratio (SNR) for proper sensing in the ADCs. In such a scenario, the need for high precision ADCs can become redundant. In summary, device variations play a crucial role in governing NVM crossbar designs. The origin of these variations has been studied in detailed for various technologies [163], [164]. Their impact, however, has been studied at an array level by simple statistical modeling. There is a need to perform detailed system-level modeling to truly understand its effects on DNN performance. Moreover, to leverage the inherent resiliency of DNNs toward erroneous computations, algorithmic techniques should be explored to overcome the effect of variations.

Besides the functional accuracy of NVM crossbars, another glaring issue that plagues the application of the technology for ML workloads is the extremely high energy and area consumption of ADCs. This effectively increases the energy consumption of the NVM crossbar, reducing its benefits over contemporary digital systems. Researchers have proposed using low precision ADCs [57], which induces approximations in the

computations and leverages the inherent resiliency of the DNNs. In addition, ML workloads have a considerable amount of sparsity in both inputs and weights. Moreover, the bit-serial nature of computations in crossbars can further increase the degree of sparsity. Previous works have focused on structured sparsity [165]–[167]. Such sparsity can reduce the ADC requirements for every array, remarkably reducing the energy consumption of NVM crossbars. Moreover, as the MVM computations are inherently approximate, ADCs with lower SNR can be sufficient for reasonably accurate conversions. Thus, ADC design is a crucial piece of the puzzle for realizing the area and energy efficiency promised by NVM crossbars. Detailed functional analysis is necessary to determine the ADC requirements considering the sparse and approximate nature of computing.

Finally, to meet the growing demands of compute and storage for future applications, NVM crossbars need to follow the scaling trends that have been a key reason for the success of CMOS systems. Scaling can be achieved by reducing the individual cell sizes as well as the reduction of interconnect and peripheral circuitry. However, different kinds of scaling have their own challenges. Device scaling can lead to more variability, while interconnect scaling may lead to an increased effect of parasitics. With careful crossbar design, larger crossbars can also lead to the sharing of peripheral circuitry, thus reducing the overall area of the chip. Moreover, mixed-signal CMOS peripherals need to be scaled effectively to improve the area efficiency of the array. Low-resistance interconnect needs to be explored to reduce the impact of parasitics. Finally, following CMOS 3-D stacking trends, NVM devices can also achieve higher density by horizontal and vertical 3-D stacking through improved lithography techniques. Such 3-D RRAM arrays have been demonstrated experimentally in the recent past. In summary, technology scaling is a key component of the development of NVM crossbars for future computing systems.

NVM crossbars have been heavily explored as approximate MVM units for ML workloads. Extensive research across the stack of devices, circuits, and architectures has exploited the rich device physics to perform MVM computations in high-density NVM arrays and build large scale systems using these analog computing cores. Despite the promises, significant effort needs to be put to overcome the challenges of variability that cripples the analog nature of computing. Furthermore, to truly deliver the promises of energy and area efficiency offered by this technology, hardware–software codesign needs to be performed. Through thorough and directed research in materials, devices, circuits, and architectures, in-memory computing with NVM crossbars can become an integral part of the future computing industry. ■

REFERENCES

- [1] D. Silver et al., "Mastering the game of Go without human knowledge," *Nature*, vol. 550, no. 7676, pp. 354–359, Oct. 2017.
- [2] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2012, pp. 1097–1105.
- [3] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.
- [4] R. Girshick, "Fast R-CNN," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Dec. 2015, pp. 1440–1448.
- [5] D. Tang, B. Qin, and T. Liu, "Document modeling with gated recurrent neural network for sentiment classification," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2015, pp. 1422–1432.
- [6] X. Xu et al., "Scaling for edge inference of deep neural networks," *Nature Electron.*, vol. 1, no. 4, pp. 216–222, Apr. 2018.
- [7] N. P. Jouppi et al., "In-datacenter performance analysis of a tensor processing unit," in *Proc. ACM/IEEE 44th Annu. Int. Symp. Comput. Archit. (ISCA)*, Jun. 2017, pp. 1–12.
- [8] E. Chung et al., "Serving DNNs in real time at datacenter scale with project brainwave," *IEEE Micro*, vol. 38, no. 2, pp. 8–20, Mar. 2018.
- [9] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, "Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks," *IEEE J. Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, Jan. 2017.
- [10] J. Lee, C. Kim, S. Kang, D. Shin, S. Kim, and H.-J. Yoo, "UNPU: A 50.6TOPS/W unified deep neural network accelerator with 1b-to-16b fully-variable weight bit-precision," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2018, pp. 218–220.
- [11] B. Moons, R. Uytterhoeven, W. Dehaene, and M. Verhelst, "14.5 envision: A 0.26-to-10TOPS/W subword-parallel dynamic-voltage-accuracy-frequency-scalable convolutional neural network processor in 28 nm FDSOI," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2017, pp. 246–247.
- [12] W. A. Wulf and S. A. McKee, "Hitting the memory wall: Implications of the obvious," *ACM SIGARCH Comput. Archit. News*, vol. 23, no. 1, pp. 20–24, Mar. 1995.
- [13] X. Si et al., "24.5 A twin-8T SRAM computation-in-memory macro for multiple-bit CNN-based machine learning," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2019, pp. 396–398.
- [14] A. Jaiswal, I. Chakraborty, A. Agrawal, and K. Roy, "8T SRAM cell as a multibit dot-product engine for beyond von Neumann computing," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 27, no. 11, pp. 2556–2567, Nov. 2019.
- [15] A. Biswas and A. P. Chandrakasan, "Conv-RAM: An energy-efficient SRAM with embedded convolution computation for low-power CNN-based machine learning applications," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2018, pp. 488–490.
- [16] M. Kang, S. K. Gonugondla, A. Patil, and N. R. Shanbhag, "A multi-functional in-memory inference processor using a standard 6T SRAM array," *IEEE J. Solid-State Circuits*, vol. 53, no. 2, pp. 642–655, Feb. 2018.
- [17] J. Zhang, Z. Wang, and N. Verma, "In-memory computation of a machine-learning classifier in a standard 6T SRAM array," *IEEE J. Solid-State Circuits*, vol. 52, no. 4, pp. 915–924, Apr. 2017.
- [18] S. Yin, Z. Jiang, J.-S. Seo, and M. Seok, "XNOR-SRAM: In-memory computing SRAM macro for binary/ternary deep neural networks," *IEEE J. Solid-State Circuits*, vol. 55, no. 6, pp. 1733–1743, Jun. 2020.
- [19] S. Yu, "Neuro-inspired computing with emerging nonvolatile memories," *Proc. IEEE*, vol. 106, no. 2, pp. 260–285, 2018.
- [20] I. Chakraborty, A. Jaiswal, A. K. Saha, S. K. Gupta, and K. Roy, "Pathways to efficient neuromorphic computing with non-volatile memory technologies," *Appl. Phys. Rev.*, vol. 7, no. 2, Jun. 2020, Art. no. 021308.
- [21] S. Ambrogio et al., "Equivalent-accuracy accelerated neural-network training using analogue memory," *Nature*, vol. 558, no. 7708, p. 60, 2018.
- [22] M. Hu et al., "Memristor-based analog computation and neural network classification with a dot product engine," *Adv. Mater.*, vol. 30, no. 9, Mar. 2018, Art. no. 1705914.
- [23] F. Cai et al., "A fully integrated reprogrammable memristor-CMOS system for efficient multiply-accumulate operations," *Nature Electron.*, vol. 2, no. 7, pp. 290–299, Jul. 2019.
- [24] A. Sengupta and K. Roy, "A vision for all-spin neural networks: A device to system perspective," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 63, no. 12, pp. 2267–2277, Dec. 2016.
- [25] D. Ielmini and H.-S.-P. Wong, "In-memory computing with resistive switching devices," *Nature Electron.*, vol. 1, no. 6, pp. 333–343, Jun. 2018.
- [26] M. A. Zidan, J. P. Strachan, and W. D. Lu, "The future of electronics based on memristive systems," *Nature Electron.*, vol. 1, no. 1, p. 22, Jan. 2018.
- [27] T. Tang, L. Xia, B. Li, Y. Wang, and H. Yang, "Binary convolutional neural network on RRAM," in *Proc. 22nd Asia South Pacific Design Autom. Conf. (ASP-DAC)*, Jan. 2017, pp. 782–787.
- [28] S. G. Ramasubramanian, R. Venkatesan, M. Sharad, K. Roy, and A. Raghunathan, "SPINDLE: SPINtronic deep learning engine for large-scale neuromorphic computing," in *Proc. Int. Symp. Low Power Electron. Design (ISLPED)*, 2014, pp. 15–20.
- [29] A. Shafee et al., "ISAAC: A convolutional neural network accelerator with in-situ analog arithmetic in crossbars," *ACM SIGARCH Comput. Archit. News*, vol. 44, no. 3, pp. 14–26, 2016.
- [30] A. Ankit et al., "PUMA: A programmable ultra-efficient memristor-based accelerator for machine learning inference," in *Proc. 24th Int. Conf. Architectural Support Program. Lang. Operating Syst.*, Apr. 2019, pp. 715–731.
- [31] J. Ambrosi et al., "Hardware-software co-design for an analog-digital accelerator for machine learning," in *Proc. IEEE Int. Conf. Rebooting Comput. (ICRC)*, 2018, pp. 1–13.
- [32] W. Liu, F. Lombardi, and M. Shulte, "A retrospective and prospective view of approximate computing [Point of View]," *Proc. IEEE*, vol. 108, no. 3, pp. 394–399, Mar. 2020.
- [33] S. Jain and A. Raghunathan, "CxDNN: Hardware-software compensation methods for deep neural networks on resistive crossbar systems," *ACM Trans. Embedded Comput. Syst.*, vol. 18, no. 6, p. 113, 2019.
- [34] I. Chakraborty, D. Roy, and K. Roy, "Technology aware training in memristive neuromorphic systems for nonideal synaptic crossbars," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 2, no. 5, pp. 335–344, Oct. 2018.
- [35] L. Ni, Y. Wang, H. Yu, W. Yang, C. Weng, and J. Zhao, "An energy-efficient matrix multiplication accelerator by distributed in-memory computing on binary RRAM crossbar," in *Proc. 21st Asia South Pacific Design Autom. Conf. (ASP-DAC)*, Jan. 2016, pp. 280–285.
- [36] A. S. Rekhi et al., "Analog/mixed-signal hardware error modeling for deep learning inference," in *Proc. 56th Annu. Design Autom. Conf.*, Jun. 2019, p. 81.
- [37] H.-S. P. Wong et al., "Phase change memory," *Proc. IEEE*, vol. 98, no. 12, pp. 2201–2227, Dec. 2010.
- [38] H.-S. P. Wong, H.-Y. Lee, S. Yu, Y.-S. Chen, Y. Wu, P.-S. Chen, B. Lee, F. T. Chen, and M.-J. Tsai, "Metal-oxide RRAM," *Proc. IEEE*, vol. 100, no. 6, pp. 1951–1970, Jun. 2012, doi: 10.1109/JPROC.2012.2190369.
- [39] X. Fong et al., "Spin-transfer torque devices for logic and memory: Prospects and perspectives," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 35, no. 1, pp. 1–22, Jan. 2016.
- [40] A. Sengupta and K. Roy, "Encoding neural and synaptic functionalities in electron spin: A pathway to efficient neuromorphic computing," *Appl. Phys. Rev.*, vol. 4, no. 4, Dec. 2017, Art. no. 041105.
- [41] S. Kim et al., "NVM neuromorphic core with 64k-cell (256-by-256) phase change memory synaptic array with on-chip neuron circuits for continuous in-situ learning," in *IEDM Tech. Dig.*, Dec. 2015, pp. 1–17.
- [42] I. Chakraborty, G. Saha, and K. Roy, "Photonic in-memory computing primitive for spiking neural networks using phase-change materials," *Phys. Rev. A, Gen. Phys. Appl.*, vol. 11, no. 1, Jan. 2019, Art. no. 014063.
- [43] G. W. Burr et al., "Experimental demonstration and tolerancing of a large-scale neural network (165 000 synapses) using phase-change memory as the synaptic weight element," *IEEE Trans. Electron Devices*, vol. 62, no. 11, pp. 3498–3507, Nov. 2015.
- [44] M. Stanisavljevic, H. Pozidis, A. Athmanathan, N. Papandreou, T. Mittelholzer, and E. Eleftheriou, "Demonstration of reliable triple-level-cell (TLC) phase-change memory," in *Proc. IEEE 8th Int. Memory Workshop (IMW)*, May 2016, pp. 1–4.
- [45] E. J. Merced-Grafals, N. Dávila, N. Ge, R. S. Williams, and J. P. Strachan, "Repeatable, accurate, and high speed multi-level programming of memristor 1T1R arrays for power efficient analog computing applications," *Nanotechnology*, vol. 27, no. 36, Sep. 2016, Art. no. 365202.
- [46] S. Ikeda et al., "Tunnel magnetoresistance of 604% at 300K by suppression of Ta diffusion in CoFeB/MgO/CoFeB pseudo-spin-valves annealed at high temperature," *Appl. Phys. Lett.*, vol. 93, no. 8, Aug. 2008, Art. no. 082508.
- [47] T. Ali et al., "A multilevel FeFET memory device based on laminated HSO and HZO ferroelectric layers for high-density storage," in *IEDM Tech. Dig.*, Dec. 2019, pp. 7–28.
- [48] Y. Watanabe et al., "Current-driven insulator-conductor transition and nonvolatile memory in chromium-doped SrTiO₃ single crystals," *Appl. Phys. Lett.*, vol. 78, no. 23, pp. 3738–3740, Jun. 2001.
- [49] L. Goux et al., "Evidences of oxygen-mediated resistive-switching mechanism in TiN/HfO₂/Pt cells," *Appl. Phys. Lett.*, vol. 97, no. 24, Dec. 2010, Art. no. 243509.
- [50] C. Rohde, B. J. Choi, D. S. Jeong, S. Choi, J.-S. Zhao, and C. S. Hwang, "Identification of a determining parameter for resistive switching of TiO₂ thin films," *Appl. Phys. Lett.*, vol. 86, no. 26, 2005, Art. no. 262907.
- [51] Z. Wei et al., "Highly reliable TaOx ReRAM and direct evidence of redox reaction mechanism," in *IEDM Tech. Dig.*, Dec. 2008, pp. 1–4.
- [52] D. Jana et al., "Conductive-bridging random access memory: Challenges and opportunity for 3D architecture," *Nanoscale Res. Lett.*, vol. 10, no. 1, p. 188, Dec. 2015.
- [53] S.-Y. Wang, C.-W. Huang, D.-Y. Lee, T.-Y. Tseng, and T.-C. Chang, "Multilevel resistive switching in Ti/Cu_xO/Pt memory devices," *J. Appl. Phys.*, vol. 108, no. 11, 2010, Art. no. 114110.
- [54] S. Park et al., "Self-formed Schottky barrier induced selector-less RRAM for cross-point memory applications," *Phys. Status Solidi (RRL)-Rapid Res. Lett.*, vol. 6, no. 11, pp. 454–456, Nov. 2012.
- [55] S. H. Jo, T. Chang, I. Ebong, B. B. Bhadviya, P. Mazumder, and W. Lu, "Nanoscale memristor device as synapse in neuromorphic systems," *Nano Lett.*, vol. 10, no. 4, pp. 1297–1301, Apr. 2010.

- [56] C. Li et al., "Analogue signal and image processing with large memristor crossbars," *Nature Electron.*, vol. 1, no. 1, p. 52, Jan. 2018.
- [57] X. Sun, S. Yin, X. Peng, R. Liu, J.-S. Seo, and S. Yu, "XNOR-RRAM: A scalable and parallel resistive synaptic architecture for binary neural networks," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Mar. 2018, pp. 1423–1428.
- [58] A. Chen and M. Lin, "Variability of resistive switching memories and its impact on crossbar array performance," in *Proc. Int. Rel. Phys. Symp.*, Monterey, CA, USA, 2011, pp. MY.7.1-MY.7.4, doi: 10.1109/IRPS.2011.5784590.
- [59] X. Guan, S. Yu, and H.-S.-P. Wong, "A SPICE compact model of metal oxide resistive switching memory with variations," *IEEE Electron Device Lett.*, vol. 33, no. 10, pp. 1405–1407, Oct. 2012.
- [60] J. C. Slonczewski, "Current-driven excitation of magnetic multilayers," *J. Magn. Magn. Mater.*, vol. 159, nos. 1–2, pp. L1–L7, Jun. 1996.
- [61] T. Kawahara et al., "2Mb spin-transfer torque RAM (SPRAM) with bit-by-bit bidirectional current write and parallelizing-direction current read," in *Proc. IEEE Int. Solid-State Circuits Conference. Dig. Tech. Papers*, Feb. 2007, pp. 480–617.
- [62] J. Mathon and A. Umerski, "Theory of tunneling magnetoresistance of an epitaxial Fe/MgO/Fe(001) junction," *Phys. Rev. B, Condens. Matter*, vol. 63, no. 22, May 2001, Art. no. 220403.
- [63] A. Hirohata et al., "Roadmap for emerging materials for spintronic device applications," *IEEE Trans. Magn.*, vol. 51, no. 10, pp. 1–11, Oct. 2015.
- [64] Y. Cassuto, S. Kvatinsky, and E. Yaakobi, "Sneak-path constraints in memristor crossbar arrays," in *Proc. IEEE Int. Symp. Inf. Theory*, Jul. 2013, pp. 156–160.
- [65] L. Liu, C.-F. Pai, Y. Li, H. W. Tseng, D. C. Ralph, and R. A. Buhrman, "Spin-torque switching with the giant spin Hall effect of tantalum," *Science*, vol. 336, no. 6081, pp. 555–558, May 2012.
- [66] E. J. Torok, A. L. Olson, and H. N. Oredson, "Transition between Bloch and Néel walls," *J. Appl. Phys.*, vol. 36, no. 4, pp. 1394–1399, Apr. 1965.
- [67] I. Dzyaloshinsky, "A thermodynamic theory of 'weak' ferromagnetism of antiferromagnetics," *J. Phys. Chem. Solids*, vol. 4, no. 4, pp. 241–255, Jan. 1958.
- [68] T. Moriya, "Anisotropic superexchange interaction and weak ferromagnetism," *Phys. Rev.*, vol. 120, no. 1, p. 91, 1960.
- [69] D. Bhowmik et al., "Deterministic domain wall motion orthogonal to current flow due to spin orbit torque," *Sci. Rep.*, vol. 5, no. 1, p. 11823, Dec. 2015.
- [70] A. J. Annunziata et al., "Racetrack memory cell array with integrated magnetic tunnel junction readout," in *IEDM Tech. Dig.*, Dec. 2011, pp. 3–24.
- [71] S. L. Miller, R. D. Nasby, J. R. Schwank, M. S. Rodgers, and P. V. Dressendorfer, "Device modeling of ferroelectric capacitors," *J. Appl. Phys.*, vol. 68, no. 12, pp. 6463–6471, Dec. 1990.
- [72] S. Dunkel et al., "A FeFET based super-low-power ultra-fast embedded NVM technology for 22 nm FDSOI and beyond," in *IEDM Tech. Dig.*, Dec. 2017, pp. 7–19.
- [73] X. Chen et al., "The impact of ferroelectric FETs on digital and analog circuits and architectures," *IEEE Des. Test. Comput.*, vol. 37, no. 1, pp. 79–99, Feb. 2020.
- [74] H. Mulaosmanovic, T. Mikolajick, and S. Slesazeck, "Accumulative polarization reversal in nanoscale ferroelectric transistors," *ACS Appl. Mater. Interfaces*, vol. 10, no. 28, pp. 23997–24002, Jul. 2018.
- [75] S. Dutta et al., "Biologically plausible ferroelectric quasi-leaky integrate and fire neuron," in *Proc. Symp. VLSI Technol.*, Jun. 2019, pp. T140–T141.
- [76] A. K. Saha and S. K. Gupta, "Modeling and comparative analysis of hysteretic ferroelectric and anti-ferroelectric FETs," in *Proc. 76th Device Res. Conf. (DRC)*, Jun. 2018, pp. 1–2.
- [77] H. Mulaosmanovic et al., "Novel ferroelectric FET based synapse for neuromorphic systems," in *Proc. Symp. VLSI Technol.*, Jun. 2017, pp. T176–T177.
- [78] A. F. Laguna, X. Yin, D. Reis, M. Niemier, and X. S. Hu, "Ferroelectric FET based in-memory computing for few-shot learning," in *Proc. Great Lakes Symp. VLSI*, May 2019, pp. 373–378.
- [79] J. Woo and S. Yu, "Impact of selector devices in analog RRAM-based crossbar arrays for inference and training of neuromorphic system," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 27, no. 9, pp. 2205–2212, Sep. 2019.
- [80] K. Gopalakrishnan et al., "Highly-scalable novel access device based on mixed ionic electronic conduction (MIEC) materials for high density phase change memory (PCM) arrays," in *Proc. Symp. VLSI Technol.*, Jun. 2010, pp. 205–206.
- [81] J. Woo, D. Lee, E. Cha, S. Lee, S. Park, and H. Hwang, "Vertically stacked ReRAM composed of a bidirectional selector and CB-RRAM for cross-point array applications," *IEEE Electron Device Lett.*, vol. 34, no. 12, pp. 1512–1514, Dec. 2013.
- [82] W. Gee Kim et al., "NbO₂-based low power and cost effective 1S1R switching for high density cross point ReRAM application," in *Symp. VLSI Technol. (VLSI-Technology)*, Dig. Tech. Papers, Jun. 2014, pp. 1–2.
- [83] R. Midya et al., "Anatomy of Ag/Hafnia-based selectors with 1010 nonlinearity," *Adv. Mater.*, vol. 29, no. 12, Mar. 2017, Art. no. 1604457. [Online]. Available: <https://onlinelibrary.wiley.com/doi/abs/10.1002/adma.201604457>
- [84] S. Hyun Jo, T. Kumar, S. Narayanan, W. D. Lu, and H. Nazarian, "3D-stackable crossbar resistive memory based on field assisted superlinear threshold (FAST) selector," in *IEDM Tech. Dig.*, Dec. 2014, pp. 1–4.
- [85] S. Yasuda et al., "A cross point Cu-ReRAM with a novel OTS selector for storage class memory applications," in *Proc. Symp. VLSI Technol.*, Jun. 2017, pp. T30–T31.
- [86] X. Peng, R. Madler, P.-Y. Chen, and S. Yu, "Cross-point memory design challenges and survey of selector device characteristics," *J. Comput. Electron.*, vol. 16, no. 4, pp. 1167–1174, Dec. 2017.
- [87] B. Razavi, "The R-2R and C-2C ladders [a circuit for all seasons]," *IEEE Solid State Circuits Mag.*, vol. 11, no. 3, pp. 10–15, Summer 2019.
- [88] C. Xue et al., "Embedded 1-mb ReRAM-based computing-in-memory macro with multibit input and weight for CNN-based AI edge processors," *IEEE J. Solid-State Circuits*, vol. 55, no. 1, pp. 203–215, Jan. 2020.
- [89] M. Hu et al., "Dot-product engine for neuromorphic computing: Programming 1T1M crossbar to accelerate matrix-vector multiplication," in *Proc. 53rd Annu. Design Autom. Conf. (DAC)*, 2016, pp. 1–6.
- [90] B. Razavi, "The current-steering DAC [a circuit for all seasons]," *IEEE Solid State Circuits Mag.*, vol. 10, no. 1, pp. 11–15, Winter 2018.
- [91] N. Cao, M. Chang, and A. Raychowdhury, "A 65-nm 8-to-3-b 1.0–0.36-V 9.1–1.1-TOPS/W hybrid-digital-mixed-signal computing platform for accelerating swarm robotics," *IEEE J. Solid-State Circuits*, vol. 55, no. 1, pp. 49–59, Jan. 2020.
- [92] B. Razavi, "The transimpedance amplifier [a circuit for all seasons]," *IEEE Solid State Circuits Mag.*, vol. 11, no. 1, pp. 10–97, Winter 2019.
- [93] B. Razavi, "A tale of two ADCs: Pipelined versus SAR," *IEEE Solid State Circuits Mag.*, vol. 7, no. 3, pp. 38–46, Summer 2015.
- [94] B. Razavi, "The flash ADC [a circuit for all seasons]," *IEEE Solid State Circuits Mag.*, vol. 9, no. 3, pp. 9–13, Summer 2017.
- [95] B. Murmann, "The successive approximation register ADC: A versatile building block for ultra-low-power to ultra-high-speed applications," *IEEE Commun. Mag.*, vol. 54, no. 4, pp. 78–83, Apr. 2016.
- [96] R. Mochida et al., "A 4M synapses integrated analog ReRAM based 66.5 TOPS/W neural-network processor with cell current controlled writing and flexible network architecture," in *Proc. IEEE Symp. VLSI Technol.*, Jun. 2018, pp. 175–176.
- [97] P. Yao et al., "Fully hardware-implemented memristor convolutional neural network," *Nature*, vol. 577, no. 7792, pp. 641–646, Jan. 2020.
- [98] Q. Liu et al., "33.2 A fully integrated analog ReRAM based 78.4TOPS/W compute-in-memory chip with fully parallel MAC computing," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2020, pp. 500–502.
- [99] Q. Wang, X. Wang, S. H. Lee, F.-H. Meng, and W. D. Lu, "A deep neural network accelerator based on tiled RRAM architecture," in *IEDM Tech. Dig.*, Dec. 2019, p. 14.
- [100] C.-X. Xue et al., "15.4 a 22 nm 2Mb ReRAM compute-in-memory macro with 121-28TOPS/W for multibit MAC computing for tiny AI edge devices," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2020, pp. 244–246.
- [101] S. Agarwal et al., "Energy scaling advantages of resistive memory crossbar based computation and its application to sparse coding," *Frontiers Neurosci.*, vol. 9, p. 484, Jan. 2016.
- [102] A. Ankit et al., "PANTHER: A programmable architecture for neural network training harnessing energy-efficient ReRAM," 2019, arXiv:1912.11516. [Online]. Available: <http://arxiv.org/abs/1912.11516>
- [103] M. J. Marinella et al., "Multiscale co-design analysis of energy, latency, area, and accuracy of a ReRAM analog neural training accelerator," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 8, no. 1, pp. 86–101, Mar. 2018.
- [104] D. Kaderotad et al., "Parallel architecture with resistive crosspoint array for dictionary learning acceleration," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 5, no. 2, pp. 194–204, Jun. 2015.
- [105] E. J. Fuller et al., "Parallel programming of an ionic floating-gate memory array for scalable neuromorphic computing," *Science*, vol. 364, no. 6440, pp. 570–574, May 2019.
- [106] K. C. Akyel et al., "DRC²: Dynamically reconfigurable computing circuit based on memory architecture," in *Proc. IEEE Int. Conf. Rebooting Comput. (ICRC)*, Oct. 2016, pp. 1–8.
- [107] M. Courbariaux, I. Hubara, D. Soudry, R. El-Yaniv, and Y. Bengio, "Binarized neural networks: Training deep neural networks with weights and activations constrained to +1 or -1," 2016, arXiv:1602.02830. [Online]. Available: <http://arxiv.org/abs/1602.02830>
- [108] M. Rastegari, V. Ordonez, J. Redmon, and A. Farhadi, "XNOR-net: ImageNet classification using binary convolutional neural networks," in *Proc. Eur. Conf. Comput. Vis.*, New York, NY, USA: Springer, 2016, pp. 525–542.
- [109] H. Valavi, P. J. Ramadge, E. Nestler, and N. Verma, "A mixed-signal binarized convolutional-neural-network accelerator integrating dense weight storage and multiplication for reduced data movement," in *Proc. IEEE Symp. VLSI Circuits*, Jun. 2018, pp. 141–142.
- [110] 2015 ITRS 2.0 Official Publication, Semicond. Ind. Assoc., San Jose, CA, USA, 2015.
- [111] D. Ielmini and Y. Zhang, "Analytical model for subthreshold conduction and threshold switching in chalcogenide-based memory devices," *J. Appl. Phys.*, vol. 102, no. 5, Sep. 2007, Art. no. 054517.
- [112] D. Ventrice, P. Fantini, A. Redaelli, A. Pirovano, A. Benvenuti, and F. Pellizzer, "A phase change memory compact model for multilevel applications," *IEEE Electron Device Lett.*, vol. 28, no. 11, pp. 973–975, Nov. 2007.
- [113] S. R. Kulkarni, D. V. Kaderotad, J.-S. Seo, and B. Rajendran, "Well-posed Verilog—A compact model for phase change memory," in *Proc. Int. Conf. Simulation Semiconductor Processes Devices (SISPAD)*, Sep. 2018, pp. 369–373.

- [114] X. Fong, S. K. Gupta, N. N. Mojumder, S. H. Chodary, C. Augustine, and K. Roy, "KNACK: A hybrid spin-charge mixed-mode simulator for evaluating different genres of spin-transfer torque MRAM bit-cells," in *Proc. Int. Conf. Simulation Semiconductor Processes Devices*, Sep. 2011, pp. 51–54.
- [115] P.-Y. Chen et al., "Mitigating effects of non-ideal synaptic device characteristics for on-chip learning," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD)*, New York, NY, USA: IEEE Press, Nov. 2015, pp. 194–199.
- [116] X. Sun and S. Yu, "Impact of non-ideal characteristics of resistive synaptic devices on implementing convolutional neural networks," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 9, no. 3, pp. 570–579, Sep. 2019.
- [117] L. Gao, P.-Y. Chen, and S. Yu, "Programming protocol optimization for analog weight tuning in resistive memories," *IEEE Electron Device Lett.*, vol. 36, no. 11, pp. 1157–1159, Nov. 2015.
- [118] C. Xu, D. Niu, S. Yu, and Y. Xie, "Modeling and design analysis of 3D vertical resistive memory—A low cost cross-point architecture," in *Proc. 19th Asia South Pacific Design Autom. Conf. (ASP-DAC)*, 2014, pp. 825–830.
- [119] S. Yu, B. Gao, Z. Fang, H. Yu, J. Kang, and H.-S.-P. Wong, "A neuromorphic visual system using RRAM synaptic devices with sub-pJ energy and tolerance to variability: Experimental characterization and large-scale modeling," in *IEDM Tech. Dig.*, Dec. 2012, pp. 4–10.
- [120] M. Zhao et al., "Characterizing endurance degradation of incremental switching in analog RRAM for neuromorphic systems," in *IEDM Tech. Dig.*, Dec. 2018, pp. 2–20.
- [121] Y. Y. Chen et al., "Improvement of data retention in HfO_2/Hf 1T1R RRAM cell under low operating current," in *IEDM Tech. Dig.*, Dec. 2013, pp. 1–10.
- [122] A. Prakash, D. Jana, and S. Maikap, "TaO_x-based resistive switching memories: Prospective and challenges," *Nanosci. Res. Lett.*, vol. 8, no. 1, p. 418, 2013.
- [123] D. Ielmini, S. Lavizzari, D. Sharma, and A. L. Lacaita, "Physical interpretation, modeling and impact on phase change memory (PCM) reliability of resistance drift due to chalcogenide structural relaxation," in *IEDM Tech. Dig.*, Dec. 2007, pp. 939–942.
- [124] A. Bosio et al., "Rebooting computing: The challenges for test and reliability," in *Proc. IEEE Int. Symp. Defect Fault Tolerance VLSI Nanotechnol. Syst. (DFT)*, Oct. 2019, pp. 8138–8143.
- [125] C. Torres-Huitzil and B. Girau, "Fault and error tolerance in neural networks: A review," *IEEE Access*, vol. 5, pp. 17322–17341, 2017.
- [126] C. Walczyk et al., "Impact of temperature on the resistive switching behavior of embedded HfO_2 -based RRAM devices," *IEEE Trans. Electron Devices*, vol. 58, no. 9, pp. 3124–3131, Sep. 2011.
- [127] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Proc. Adv. Neural Inf. Process. Syst.* New York, NY, USA: Curran Associates, 2019, pp. 8024–8035. [Online]. Available: <http://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.pdf>
- [128] M. Abadi et al., "Tensorflow: A system for large-scale machine learning," in *Proc. 12th USENIX Symp. Operating Syst. Design Implement. (OSDI)*, 2016, pp. 265–283.
- [129] Y. Jia et al., "Caffe: Convolutional architecture for fast feature embedding," in *Proc. ACM Int. Conf. Multimedia (MM)*, 2014, pp. 675–678.
- [130] S. Jain, A. Sengupta, K. Roy and A. Raghunathan, "RxnN: A framework for evaluating deep neural networks on resistive crossbars," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, to be published, doi: 10.1109/TCAD.2020.3000185.
- [131] Y. Jeong, M. A. Zidan, and W. D. Lu, "Parasitic effect analysis in memristor-array-based neuromorphic systems," *IEEE Trans. Nanotechnol.*, vol. 17, no. 1, pp. 184–193, Jan. 2018.
- [132] I. Chakraborty, M. Fayed Ali, D. Eun Kim, A. Ankit, and K. Roy, "GENIE: A generalized approach to emulating non-ideality in memristive Xbars using neural networks," 2020, *arXiv:2003.06902*. [Online]. Available: <http://arxiv.org/abs/2003.06902>
- [133] S. Han, J. Pool, J. Tran, and W. Dally, "Learning both weights and connections for efficient neural network," in *Proc. Adv. Neural Inf. Process. Syst.*, 2015, pp. 1135–1143.
- [134] S. G. Narendra, L. C. Fujino, and K. C. Smith, "Through the looking glass? The 2015 edition: Trends in solid-state circuits from ISSCC," *IEEE Solid State Circuits Mag.*, vol. 7, no. 1, pp. 14–24, Winter 2015.
- [135] X. Liu et al., "RENO: A high-efficient reconfigurable neuromorphic computing accelerator design," in *Proc. 52nd ACM/EDAC/IEEE Design Autom. Conf. (DAC)*, Jun. 2015, pp. 1–6.
- [136] P. Chi et al., "PRIME: A novel processing-in-memory architecture for neural network computation in ReRAM-based main memory," in *Proc. ACM/IEEE 43rd Annu. Int. Symp. Comput. Archit. (ISCA)*, Jun. 2016, pp. 27–39.
- [137] A. Ankit, A. Sengupta, P. Panda, and K. Roy, "RESPARC: A reconfigurable and energy-efficient architecture with memristive crossbars for deep spiking neural networks," in *Proc. 54th Annu. Design Autom. Conf.*, Jun. 2017, p. 27.
- [138] Y. Kim, Y. Zhang, and P. Li, "A reconfigurable digital neuromorphic processor with memristive synaptic crossbar for cognitive computing," *ACM J. Emerg. Technol. Comput. Syst.*, vol. 11, no. 4, p. 38, Apr. 2015.
- [139] M. N. Bojnordi and E. Ipek, "Memristive Boltzmann machine: A hardware accelerator for combinatorial optimization and deep learning," in *Proc. 5th Berkeley Symp. Energy Efficient Electron. Syst. Steep Transistors Workshop (E S)*, Oct. 2017, pp. 1–13.
- [140] S. Agarwal et al., "Resistive memory device requirements for a neural algorithm accelerator," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Jul. 2016, pp. 929–938.
- [141] P. Narayanan et al., "Toward on-chip acceleration of the backpropagation algorithm using nonvolatile memory," *IBM J. Res. Develop.*, vol. 61, nos. 4–5, pp. 1–11, 2017.
- [142] Y. Kim, H. Kim, D. Ahn, and J.-J. Kim, "Input-splitting of large neural networks for power-efficient accelerator with resistive crossbar memory array," in *Proc. Int. Symp. Low Power Electron. Design*, Jul. 2018, p. 41.
- [143] Z. Chen and B. Liu, "Lifelong machine learning," *Synth. Lectures Artif. Intell. Mach. Learn.*, vol. 10, no. 3, pp. 1–145, Nov. 2016.
- [144] B. C. Stadie, S. Levine, and P. Abbeel, "Incentivizing exploration in reinforcement learning with deep predictive models," 2015, *arXiv:1507.00814*. [Online]. Available: <http://arxiv.org/abs/1507.00814>
- [145] A. Nag et al., "Newton: Gravitating towards the physical limits of crossbar acceleration," *IEEE Micro*, vol. 38, no. 5, pp. 41–49, Sep. 2018.
- [146] B. Reagen et al., "Ares: A framework for quantifying the resilience of deep neural networks," in *Proc. 55th ACM/ESDA/IEEE Design Autom. Conf. (DAC)*, Jun. 2018, pp. 1–6.
- [147] N. Zmora et al., *Neural Network Distiller*. Accessed: Jun. 2018. [Online]. Available: <https://doi.org/10.5281/zenodo.1297430>
- [148] P.-Y. Chen, X. Peng, and S. Yu, "NeuroSim: A circuit-level macro model for benchmarking neuro-inspired architectures in online learning," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 37, no. 12, pp. 3067–3080, Dec. 2018.
- [149] S. Jain et al., "Neural network accelerator design with resistive crossbars: Opportunities and challenges," *IBM J. Res. Develop.*, vol. 63, no. 6, pp. 1–10, 2019.
- [150] P. Mickevicius et al., "Mixed precision training," 2017, *arXiv:1710.03740*. [Online]. Available: <http://arxiv.org/abs/1710.03740>
- [151] S. Roy, S. Sridharan, S. Jain, and A. Raghunathan, "TxSim: Modeling training of deep neural networks on resistive crossbar systems," 2020, *arXiv:2002.11151*. [Online]. Available: <http://arxiv.org/abs/2002.11151>
- [152] S. Agarwal et al., *CrossSim*. Accessed: Jun. 30, 2020. [Online]. Available: <http://cross-sim.sandia.gov>
- [153] C. Liu, M. Hu, J. P. Strachan, and H. Li, "Rescuing memristor-based neuromorphic design with high defects," in *Proc. 54th Annu. Design Autom. Conf.*, Jun. 2017, pp. 1–6.
- [154] B. Liu et al., "Reduction and IR-drop compensations techniques for reliable neuromorphic computing systems," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD)*, New York, NY, USA: IEEE Press, Nov. 2014, pp. 63–70.
- [155] B. Liu, H. Li, Y. Chen, X. Li, Q. Wu, and T. Huang, "Vortex: Variation-aware training for memristor X-bar," in *Proc. 52nd Annu. Design Autom. Conf. (DAC)*, 2015, pp. 1–6.
- [156] A. Agrawal, C. Lee, and K. Roy, "X-CHANGR: Changing memristive crossbar mapping for mitigating line-resistance induced accuracy degradation in deep neural networks," 2019, *arXiv:1907.00285*. [Online]. Available: <http://arxiv.org/abs/1907.00285>
- [157] L. Chen et al., "Accelerator-friendly neural-network training: Learning variations and defects in RRAM crossbar," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Mar. 2017, pp. 19–24.
- [158] M. J. Rasch, T. Gokmen, and W. Haensch, "Training large-scale ANNs on simulated resistive crossbar arrays," 2019, *arXiv:1906.02698*. [Online]. Available: <http://arxiv.org/abs/1906.02698>
- [159] T. Gokmen, M. Onen, and W. Haensch, "Training deep convolutional neural networks with resistive cross-point devices," *Frontiers Neurosci.*, vol. 11, p. 538, Oct. 2017.
- [160] S. Yu, "Resistive random access memory (RRAM)," *Synth. Lectures Emerg. Eng. Technol.*, vol. 2, no. 5, pp. 1–79, Mar. 2016.
- [161] Y. Xie, *Emerging Memory Technologies: Design, Architecture, and Applications*. New York, NY, USA: Springer, 2013.
- [162] A. Pirovano, A. L. Lacaita, F. Pellizzer, S. A. Kostylev, A. Benvenuti, and R. Bez, "Low-field amorphous state resistance and threshold voltage drift in chalcogenide materials," *IEEE Trans. Electron Devices*, vol. 51, no. 5, pp. 714–719, May 2004.
- [163] D. Ielmini, A. L. Lacaita, and D. Mantegazza, "Recovery and drift dynamics of resistance and threshold voltages in phase-change memories," *IEEE Trans. Electron Devices*, vol. 54, no. 2, pp. 308–315, Feb. 2007.
- [164] S. Ambrogio, S. Balatti, A. Cubeta, A. Calderoni, N. Ramaswamy, and D. Ielmini, "Statistical fluctuations in HfO_x resistive-switching memory: Part II—Random telegraph noise," *IEEE Trans. Electron Devices*, vol. 61, no. 8, pp. 2920–2927, Aug. 2014.
- [165] A. Ankit, A. Sengupta, and K. Roy, "TraNNsformer: Neural network transformation for memristive crossbar based neuromorphic system design," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD)*, New York, NY, USA: IEEE Press, Nov. 2017, pp. 533–540.
- [166] Y. Wang, W. Wen, B. Liu, D. Chiarulli, and H. Li, "Group scissor: Scaling neuromorphic computing design to large neural networks," in *Proc. 54th Annu. Design Autom. Conf.*, Jun. 2017, pp. 1–6.
- [167] L. Liang et al., "Crossbar-aware neural network pruning," *IEEE Access*, vol. 6, pp. 58324–58337, 2018.

ABOUT THE AUTHORS

Indranil Chakraborty (Graduate Student Member, IEEE) received the B.Engg. degree in electronics and telecommunication engineering from Jadavpur University, Kolkata, India, in 2013, and the M.Tech. degree in electrical engineering from IIT Bombay, Mumbai, India, in 2016. He is working toward the Ph.D. degree at the Nanoelectronics Research Laboratory, Purdue University, West Lafayette, IN, USA.



His primary research interests lie in hardware for machine-learning using complementary metal-oxide-semiconductor (CMOS) and emerging technologies.

Mr. Chakraborty was a recipient of the Best M.Tech. Thesis Award and the Academic Excellence Award during his time at IIT Bombay for his academic performance.

Shubham Jain received the B.Tech. degree (Hons.) in electronics and electrical communication engineering from IIT Kharagpur, Kharagpur, India, in 2012, and the Ph.D. degree in electrical and computer engineering from Purdue University, West Lafayette, IN, USA, in 2019.



He was a Design Engineer with the Bangalore Design Center, Qualcomm, Bengaluru, India, from 2012 to 2014. He was a Summer Intern with the IBM Thomas J. Watson Research Center, Yorktown Heights, NY, USA, from 2017 to 2018, where he is currently a Research Staff Member. His primary research interests include AI hardware, architecture for post-complementary metal-oxide-semiconductor (CMOS) devices, in-memory computing, and approximate computing.

Dr. Jain received the Mitacs Globalink Scholarship from Mitacs in 2011, the Andrews Fellowship from Purdue University in 2014, and the A. Richard Newton Young Student Fellowship from the Design Automation Conference (DAC) in 2015. His research has received the Best Technical Paper Award at DAC 2018 and the Best-In Session Award at TECHCON 2016.

Mustafa Ali (Graduate Student Member, IEEE) received the B.Sc. and M.Sc. degrees in electrical engineering from the Military Technical College (MTC), Cairo, Egypt, in 2011 and 2016, respectively. He is working toward the Ph.D. degree at the Nano-Electronics Research Lab (NRL), Purdue University, West Lafayette, IN, USA, under the guidance of Prof. Roy.



He worked on flexible electronics applications using TFTs in his M.Sc. degree from 2014 to 2016. He was a TA and an RA with MTC from 2013 to 2017. He was a Hardware and Embedded Systems Engineer with Integreight, Inc., Cairo, from 2012 to 2017. He joined the Nanoelectronics Research Lab (NRL), Purdue University, in spring 2018. His research interest lies in accelerating brain-inspired computing and machine learning. He is also interested in in-memory computing based on complementary metal-oxide-semiconductor (CMOS) and post-CMOS devices.

Mr. Ali achieved the First Rank in undergraduate in MTC in 2011. He was honored with the Duty Medal for excellent performance during his studies.

Sourjya Roy (Graduate Student Member, IEEE) received the bachelor's degree (Hons.) from Purdue University, West Lafayette, IN, USA, in 2018, where he is working toward the Ph.D. degree at the School of Electrical and Computer Engineering under the guidance of Prof. Anand Raghunathan.



He participated in the Summer Undergraduate Research Fellowship (SURF) at Purdue University in 2016. His current research interests include circuit and architecture codesign for complementary metal-oxide-semiconductor (CMOS), post-CMOS devices, in-memory computation, energy-efficient hardware for machine learning (ML) workloads, and algorithms to support the underlying energy-efficient hardware.

Mr. Roy was a recipient of the Charles W Brown ECE Scholarship and the Ross Fellowship from Purdue University in 2018 and 2019.

Aayush Ankit (Member, IEEE) received the B.Tech. degree from IIT (BHU) Varanasi, Varanasi, India, in 2015. He is working toward the Ph.D. degree in electrical and computer engineering at Purdue University, West Lafayette, IN, USA.



During his Ph.D. degree, he has done internships as an ML Architect at HPE Labs, Palo Alto, CA, USA, in 2017, a CPU Designer at Intel Corporation, Hillsboro, OR, USA, in 2017, and a GPU Architect at Samsung ACL, San Jose, CA, USA, in 2019. He has been a Research Assistant to Prof. Kaushik Roy at Purdue University since fall 2015. His research interests lie in hardware-software code-signs for efficient machine learning and GPU architecture-compiler designs.

Shrihari Sridharan (Graduate Student Member, IEEE) received the B.S. degree in electrical and computer engineering from Purdue University, West Lafayette, IN, USA, in 2018, where he is working toward the Ph.D. degree at the Center for Brain-Inspired Computing (C-BRIC).



His research interests include designing efficient algorithms and architectures for deep learning and other cognitive systems using post-complementary metal-oxide-semiconductor (CMOS) technologies.

Mr. Sridharan was a recipient of the Summer Undergraduate Research Fellowship (SURF) to work as Research Intern at the Nanoelectronics Research Laboratory, Purdue University in 2016.

Amogh Agrawal received the B.Tech. degree in electrical engineering from IIT Ropar, Rupnagar, India, in 2016. He is working toward the Ph.D. degree at Purdue University, West Lafayette, IN, USA, under the guidance of Prof. Kaushik Roy.



He was a Research Intern with the University of Ulm, Ulm, Germany, in 2015, under the DAAD (German Academic Exchange Service) Fellowship. In 2016, he joined the Nanoelectronics Research Lab, Purdue University. He was a Technology Development Intern with GLOBALFOUNDRIES, Malta, NY, USA, during summer 2018. His primary research interests include modeling and simulation of spin devices for application in logic, memories, and neuromorphic computing. He is also looking at in-memory computing techniques for neuromorphic computing using complementary metal-oxide-semiconductor (CMOS) and beyond-CMOS memories.

Dr. Agrawal was awarded the Directors Gold Medal for his all-round performance and the University Silver Medal for his academic achievements at IIT Ropar. He was a recipient of the Andrews Fellowship from Purdue University in 2016.

Anand Raghunathan (Fellow, IEEE) received the B.Tech. degree from IIT Madras, Chennai, India, and the M.A. and Ph.D. degrees from Princeton University, Princeton, NJ, USA.



He was a Senior Research Staff Member with NEC Laboratories America, Princeton, NJ, USA, where he led projects on system-on-chip architecture and design methodology. He has also held the Gopalakrishnan Visiting Chair at the Department of Computer Science and Engineering, IIT Madras. He is currently a Professor of electrical and computer engineering and the Chair of the VLSI with Purdue University, West Lafayette, IN, USA, where he directs the research with the Integrated Systems Laboratory. He has coauthored a book, eight book chapters, and over 200 refereed journal articles and conference papers. He holds 21 U.S. patents. His current research interests include domain-specific architecture, system-on-chip design, computing with post-complementary metal-oxide-semiconductor (CMOS) devices, and heterogeneous parallel computing.

Dr. Raghunathan is a Golden Core Member of the IEEE Computer Society. He has been a member of the technical program and organizing committees of several leading conferences and workshops. His publications received eight best paper awards and five best paper nominations. He received the Patent of the Year Award and two technology commercialization awards from NEC. He also received the IEEE Meritorious Service Award and the Outstanding Service Award. He was chosen among the Massachusetts Institute of Technology (MIT) TR35 (top 35 innovators under 35 years across various disciplines of science and technology) in 2006. He chaired premier IEEE/ACM conferences [such as the International Conference on Compilers, Architecture, and Synthesis for Embedded Systems (CASES), the International Symposium on Low Power Electronics and Design (ISLPED), the VLSI Test Symposium (VTS), and the VLSI Design]. He has served on the editorial boards of various IEEE and ACM journals in his areas of interest.

Kaushik Roy (Fellow, IEEE) received the B.Tech. degree in electronics and electrical communications engineering from IIT Kharagpur, Kharagpur, India, in 1983, and the Ph.D. degree from the Electrical and Computer Engineering Department, University of Illinois at Urbana-Champaign, Champaign, IL, USA, in 1990.



He was with the Semiconductor Process and Design Center, Texas Instruments, Dallas, TX, USA, where he worked on FPGA architecture development and low-power circuit design. He joined the Electrical and Computer Engineering Faculty, Purdue University, West Lafayette, IN, USA, in 1993, where he is currently the Edward G. Tiedemann Jr. Distinguished Professor, and the Director of the Center for Brain-Inspired Computing (C-BRIC) funded by SRC/DARPA. He was a Purdue University Faculty Scholar from 1998 to 2003. He was a Research Visionary Board Member of Motorola Labs in 2002 and held the M. Gandhi Distinguished Visiting Faculty at IIT Bombay, Mumbai, India, and the Global Foundries Visiting Chair at the National University of Singapore, Singapore. He has published more than 700 articles in refereed journals and conferences, holds 18 patents, supervised 85 Ph.D. dissertations, and is a coauthor of two books on low power complementary metal-oxide-semiconductor (CMOS) VLSI design (John Wiley & McGraw Hill). His research interests include neuromorphic and emerging computing models, neuromimetic devices, spintronics, device-circuit-algorithm codesign for nanoscale silicon and nonsilicon technologies, and low-power electronics.

Dr. Roy was the DoD Vannevar Bush Faculty Fellow from 2014 to 2019. He received the National Science Foundation Career Development Award in 1995, the IBM Faculty Partnership Award, the ATT/Lucent Foundation Award, the 2005 SRC Technical Excellence Award, the SRC Inventors Award, the Purdue College of Engineering Research Excellence Award, the Humboldt Research Award in 2010, the 2010 IEEE Circuits and Systems Society Technical Achievement Award (Charles Desoer Award), the Distinguished Alumnus Award from IIT Kharagpur, the Semiconductor Research Corporation Aristotle Award in 2015, the Best Paper Awards at the 1997 International Test Conference, the 2000 IEEE International Symposium on Quality of IC Design, the 2003 IEEE Latin American Test Workshop, the 2003 IEEE Nano, the 2004 IEEE International Conference on Computer Design, the 2006 IEEE/ACM International Symposium on Low Power Electronics & Design, and the 2005 IEEE Circuits and System Society Outstanding Young Author Award (Chris Kim), the 2006 IEEE Transactions on VLSI Systems Best Paper Award, the 2012 ACM/IEEE International Symposium on Low Power Electronics and Design Best Paper Award, and the 2013 IEEE Transactions on VLSI Best Paper Award. He was the Fulbright-Nehru Distinguished Chair. He has been on the Editorial Board of *IEEE Design & Test*, the IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS, the IEEE TRANSACTIONS ON VERY LARGE SCALE INTEGRATION (VLSI) SYSTEMS, and the IEEE TRANSACTIONS ON ELECTRON DEVICES. He was a Guest Editor for the Special Issue on Low-Power VLSI in *IEEE Design & Test* in 1994, the IEEE TRANSACTIONS ON VERY LARGE SCALE INTEGRATION (VLSI) SYSTEMS in June 2000, the *IEEE Proceedings—Computers and Digital Techniques* in July 2002, and the IEEE JOURNAL ON EMERGING AND SELECTED TOPICS IN CIRCUITS AND SYSTEMS in 2011.